

Radix Sorting

FOR MANY SORTING applications, the keys used to define the order of the records for files can be complicated. For example, consider the complex nature of the keys used in a telephone book or a library catalog. To separate this complication from essential properties of the sorting methods that we have been studying, we have used just the basic operations of comparing two keys and exchanging two records (hiding all the details of manipulating keys in these functions) as the abstract interface between sorting methods and applications for most of the methods in Chapters 6 through 9. In this chapter, we examine a different abstraction for sort keys. For example, processing the full key at every step is often unnecessary: to look up a person's number in a telephone book, we often just check the first few letters in the name to find the page containing the number. To gain similar efficiencies in sorting algorithms, we shall shift from the abstract operation where we compare keys to an abstraction where we decompose keys into a sequence of fixed-sized pieces, or *bytes*. Binary numbers are sequences of bits, strings are sequences of characters, decimal numbers are sequences of digits, and many other (but not all) types of keys can be viewed in this way. Sorting methods built on processing numbers one piece at a time are called *radix sorts*. These methods do not just compare keys: They process and compare pieces of keys.

Radix-sorting algorithms treat the keys as numbers represented in a base- R number system, for various values of R (the *radix*), and work with individual digits of the numbers. For example, when a machine at the post office processes a pile of packages that have on them five-digit decimal numbers, it distributes the packages into ten piles: one having

numbers beginning with 0, one having numbers beginning with 1, one having numbers beginning with 2, and so forth. If necessary, the piles can be processed individually, by using the same method on the next digit or by using some easier method if there are only a few packages. If we were to pick up the packages in the piles in order from 0 to 9 and in order within each pile after they have been processed, we would get them in sorted order. This procedure is a simple example of a radix sort with $R = 10$, and it is the method of choice in many real sorting applications where keys are 5- to 10-digit decimal numbers, such as postal codes, telephone numbers or social-security numbers. We shall examine the method in detail in Section 10.3.

Different values of the radix R are appropriate in various applications. In this chapter, we focus primarily on keys that are integers or strings, where radix sorts are widely used. For integers, because they are represented as binary numbers in computers, we most often work with $R = 2$ or some power of 2, because this choice allows us to decompose keys into independent pieces. For keys that involve strings of characters, we use $R = 128$ or $R = 256$, aligning the radix with the byte size. Beyond such direct applications, we can ultimately treat virtually *anything* that is represented inside a digital computer as a binary number, and we can recast many sorting applications using other types of keys to make feasible the use of radix sorts operating on keys that are binary numbers.

Radix-sorting algorithms are based on the abstract operation “extract the i th digit from a key.” Fortunately, C provides low-level operators that make it possible to implement such an operation in a straightforward and efficient manner. This fact is significant because many other languages (for example, Pascal), to encourage us to write machine-independent programs, intentionally make it difficult to write a program that depends on the way that a particular machine represents numbers. In such languages, it is difficult to implement many types of bit-by-bit manipulation techniques that actually suit most computers well. Radix sorting in particular was, for a time, a casualty of this “progressive” philosophy. But the designer of C recognized that direct manipulation of bits is often useful, and we shall be able to take advantage of C’s low-level facilities to implement radix sorts.

Good hardware support also is required; and it cannot be taken for granted. Some machines (both old and new) provide efficient ways

to get at small data, but some other machines (both old and new) slow down significantly when such operations are used. Whereas radix sorts are simply expressed in terms of the extract-the-digit operation, the task of getting peak performance out of a radix sorting algorithm can be a fascinating introduction to our hardware and software environment.

There are two, fundamentally different, basic approaches to radix sorting. The first class of methods involves algorithms that examine the digits in the keys in a left-to-right order, working with the most significant digits first. These methods are generally referred to as *most-significant-digit (MSD) radix sorts*. MSD radix sorts are attractive because they examine the minimum amount of information necessary to get a sorting job done (see Figure 10.1). MSD radix sorts generalize quicksort, because they work by partitioning the file to be sorted according to the leading digits of the keys, then recursively applying the same method to the subfiles. Indeed, when the radix is 2, we implement MSD radix sorting in a manner similar to that for quicksort. The second class of radix-sorting methods is different: They examine the digits in the keys in a right-to-left order, working with the least significant digits first. These methods are generally referred to as *least-significant-digit (LSD) radix sorts*. LSD radix sorts are somewhat counterintuitive, since they spend processing time on digits that cannot affect the result, but it is easy to ameliorate this problem, and this venerable approach is the method of choice for many sorting applications.

10.1 Bits, Bytes, and Words

The key to understanding radix sort is to recognize that (i) computers generally are built to process bits in groups called *machine words*, which are often grouped into smaller pieces call *bytes*; (ii) sort keys *also* are commonly organized as byte sequences; and (iii) small byte sequences can also serve as array indices or machine addresses. Therefore, it will be convenient for us to work with the following abstractions.

.396465048	.015583409	.0
.353336658	.159072306	.1590
.318693642	.159369371	.1593
.015583409	.269971047	.2
.159369371	.318693642	.31
.691004885	.353336658	.35
.899854354	.396465048	.39
.159072306	.538069659	.5
.604144269	.604144269	.60
.269971047	.691004885	.69
.538069659	.899854354	.8

Figure 10.1
MSD radix sorting

Even though the 11 numbers between 0 and 1 on this list (left) each have nine digits for a total of 99 digits, we can put them in order (center) by just examining 22 of the digits (right).

Definition 10.1 *A byte is a fixed-length sequence of bits; a string is a variable-length sequence of bytes; a word is a fixed-length sequence of bytes.*

In radix sorting, depending on the context, a *key* may be a word or a string. Some of the radix-sorting algorithms that we consider in this chapter depend on the keys being fixed length (words); others are designed to adapt to the situation when the keys are variable length (strings).

A typical machine might have 8-bit bytes and 32- or 64-bit words (the actual values may be found in the header file `<limits.h>`), but it will be convenient for us to consider various other byte and word sizes as well (generally small integer multiples or fractions of built-in machine sizes). We use machine- and application-dependent defined constants for the number of bits per word and the number of bits per byte:

```
#define bitsword 32
#define bitsbyte 8
#define bytesword 4
#define R (1 << bitsbyte)
```

Also included in these definitions for use when we begin looking at radix sorts is the constant `R`, the number of different byte values. When using these definitions, we generally assume that `bitsword` is a multiple of `bitsbyte`; that the number of bits per machine word is not less than (typically, is equal to) `bitsword`; and that bytes are individually addressable. Different computers have different conventions for referring to their bits and bytes; for the purposes of our discussion, we will consider the bits in a word to be numbered, left to right, from 0 to `bitsword-1`, and the bytes in a word to be numbered, left to right, from 0 to `bytesword-1`. In both cases, we assume the numbering to also be from most significant to least significant.

Most computers have bitwise *and* and *shift* operations, which we can use to extract bytes from words. In C, we can directly express the operation of extracting the *B*th byte of a binary word *A* as follows:

```
#define digit(A, B)
(((A) >> (bitsword-((B)+1)*bitsbyte)) & (R-1))
```

For example, this macro would extract byte 2 (the third byte) of a 32-bit number by shifting right $32 - 3 * 8 = 8$ bit positions, then using the

mask 00000000000000000000000011111111 to zero out all the bits except those of the desired byte, in the 8 bits at the right.

Another option on many machines is to arrange things such that the radix is aligned with the byte size, and therefore a single access will get the right bits quickly. This operation is supported directly for strings in C:

```
#define digit(A, B) A[B].
```

This approach could be used for numbers as well, though differing number-representation schemes may make such code nonportable. In any case, we need to be aware that byte-access operations of this type might be implemented with underlying shift-and-mask operations similar to the ones in the previous paragraph in some computing environments.

At a slightly different level of abstraction, we can think of keys as numbers and bytes as digits. Given a (key represented as a) number, the fundamental operation needed for radix sorts is to extract a digit from the number. When we choose a radix that is a power of 2, the digits are groups of bits, which we can easily access directly using one of the macros just discussed. Indeed, the primary reason that we use radices that are powers of 2 is that the operation of accessing groups of bits is inexpensive. In some computing environments, we can use other radices, as well. For example, if a is a positive integer, the b th digit (from the right) of the radix- R representation of a is

$$\lfloor a/R^b \rfloor \bmod R.$$

On a machine built for high-performance numerical calculations, this computation might be as fast for general R as for $R = 2$.

Yet another viewpoint is to think of keys as numbers between 0 and 1 with an implicit decimal point at the left, as shown in Figure 10.1. In this case, the b th digit (from the left) of a is

$$\lfloor aR^b \rfloor \bmod R.$$

If we are using a machine where we can do such operations efficiently, then we can use them as the basis for our radix sort. This model applies when keys are variable length, such as character strings.

Thus, for the remainder of this chapter, we view keys as radix- R numbers (with R not specified), and use the abstract `digit` operation

to access digits of keys, with confidence that we will be able to develop fast implementations of `digit` for particular computers.

Definition 10.2 *A key is a radix- R number, with digits numbered from the left (starting at 0).*

In light of the examples that we just considered, it is safe for us to assume that this abstraction will admit efficient implementations for many applications on most computers, although we must be careful that a particular implementation is efficient within a given hardware and software environment.

We assume that the keys are not short, so it is worthwhile to extract their bits. If the keys are short, then we can use the key-indexed counting method of Chapter 6. Recall that this method can sort N keys known to be integers between 0 and $R - 1$ in linear time, using one auxiliary table of size R for counts and another of size N for rearranging records. Thus, if we can afford a table of size 2^w , then w -bit keys can easily be sorted in linear time. Indeed, key-indexed counting lies at the heart of the basic MSD and LSD radix-sorting methods. Radix sorting comes into play when the keys are sufficiently long (say $w = 64$) that using a table of size 2^w is not feasible.

Exercises

- ▷ 10.1 How many digits are there when a 32-bit quantity is viewed as a radix-256 number? Describe how to extract each of the digits. Answer the same question for radix 2^{16} .
- ▷ 10.2 For $N = 10^3$, 10^6 , and 10^9 , give the smallest byte size that allows any number between 0 and N to be represented in a 4-byte word.
- 10.3 Implement a `less` function using the `digit` abstraction (so that, for example, we could run empirical studies comparing the algorithms in Chapters 6 and 9 with the methods in this chapter, using the same data).
- 10.4 Design and carry out an experiment to compare the cost of extracting digits using bit-shifting and arithmetic operations on your machine. How many digits can you extract per second, using each of the two methods? *Note:* Be wary; your compiler might convert arithmetic operations to bit-shifting ones, or vice versa!
- 10.5 Write a program that, given a set of N random decimal numbers ($R = 10$) uniformly distributed between 0 and 1, will compute the number of digit comparisons necessary to sort them, in the sense illustrated in Figure 10.1. Run your program for $N = 10^3$, 10^4 , 10^5 , and 10^6 .
- 10.6 Answer Exercise 10.5 for $R = 2$, using random 32-bit quantities.

- 10.7 Answer Exercise 10.5 for the case where the numbers are distributed according to a Gaussian distribution.

10.2 Binary Quicksort

Suppose that we can rearrange the records of a file such that all those whose keys begin with a 0 bit come before all those whose keys begin with a 1 bit. Then, we can use a recursive sorting method that is a variant of quicksort (see Chapter 7): Partition the file in this way, then sort the two subfiles independently. To rearrange the file, scan from the left to find a key that starts with a 1 bit, scan from the right to find a key that starts with a 0 bit, exchange, and continue until the scanning pointers cross. This method is often called *radix-exchange sort* in the literature (including in earlier editions of this book); here, we shall use the name *binary quicksort* to emphasize that it is a simple variant of the algorithm invented by Hoare, even though it was actually discovered before quicksort was (*see reference section*).

Program 10.1 is a full implementation of this method. The partitioning process is essentially the same as Program 7.2, except that the number 2^b , instead of some key from the file, is used as the partitioning element. Because 2^b may not be in the file, there can be no guarantee that an element is put into its final place during partitioning. The algorithm also differs from normal quicksort because the recursive calls are for keys with 1 fewer bit. This difference has important implications for performance. For example, when a degenerate partition occurs for a file of N elements, a recursive call for a subfile of size N will result, for keys with 1 fewer bit. Thus, the number of such calls is limited by the number of bits in the keys. By contrast, consistent use of partitioning values not in the file in a standard quicksort could result in an infinite recursive loop.

As there are with standard quicksort, various options are available in implementing the inner loop. In Program 10.1, tests for whether the pointers have crossed are included in both inner loops. This arrangement results in an extra exchange for the case $i = j$, which could be avoided with a `break`, as is done in Program 7.2, although in this case the exchange of `a[i]` with itself is harmless. Another alternative is to use sentinel keys.

```

A S O R T I N G E X A M P L E
A E O L M I N G E A X T P R S
                                S T P R X
                                S R P T
                                P R S
                                R S

A E A E G I N M L O
                                I N M L O
                                L M N O
                                N O
                                L M

A A E E G
  E E G
  E E
  E E

A A
A A
A A
A A E E G I L M N O P R S T X

```

Figure 10.2
Binary quicksort example

Partitioning on the leading bit does not guarantee that one value will be put into place; it guarantees only that all keys with leading 0 bits come before all keys with leading 1 bits. We can compare this diagram with Figure 7.1 for quicksort, although the operation of the partitioning method is completely opaque without the binary representation of the keys. Figure 10.3 gives the details that explain the partition positions precisely.

Program 10.1 Binary quicksort

This program partitions a file on the leading bits of the keys, and then sorts the subfiles recursively. The variable *w* keeps track of the bit being examined, starting at 0 (leftmost). The partitioning stops with *j* equal to *i*, and all elements to the right of *a[i]* having 1 bits in the *w*th position and all elements to the left of *a[i]* having 0 bits in the *w*th position. The element *a[i]* itself will have a 1 bit *unless* all keys in the file have a 0 in position *w*. An extra test just after the partitioning loop covers this case.

```
quicksortB(int a[], int l, int r, int w)
{
    int i = l, j = r;
    if (r <= l || w > bitsword) return;
    while (j != i)
    {
        while (digit(a[i], w) == 0 && (i < j)) i++;
        while (digit(a[j], w) == 1 && (j > i)) j--;
        exch(a[i], a[j]);
    }
    if (digit(a[r], w) == 0) j++;
    quicksortB(a, l, j-1, w+1);
    quicksortB(a, j, r, w+1);
}

void sort(Item a[], int l, int r)
{
    quicksortB(a, l, r, 0);
}
```

Figure 10.2 depicts the operation of Program 10.1 on a small sample file, for comparison with Figure 7.1 for quicksort. This figure shows what the data movement is, but not *why* the various moves are made—that depends on the binary representation of the keys. A more detailed view for the same example is given in Figure 10.3. This example assumes that the letters are encoded with a simple 5-bit code, with the *i*th letter of the alphabet represented by the binary representation of the number *i*. This encoding is a simplified version of real character codes, which use more bits (7, 8, or even 16) to represent more characters (uppercase or lowercase letters, numbers, and special symbols).

A	00001	A	00001	A	00001	A	00001	A	00001	A	00001
S	10011	E	00101	E	00101	A	00001	A	00001	A	00001
O	01111	O	01111	A	00001	E	00101	E	00101	E	00101
R	10010	L	01100	E	00101	E	00101	E	00101	E	00101
T	10100	M	01101	G	00111	G	00111	G	00111	G	00111
I	01001	I	01001	I	01001	I	01001	I	01001	I	01001
N	01110	N	01110	N	01110	N	01110	L	01100	L	01100
G	00111	G	00111	M	01101	M	01101	M	01101	M	01101
E	00101	E	00101	L	01100	L	01100	N	01110	N	01110
X	11000	A	00001	O	01111	O	01111	O	01111	O	01111
A	00001	X	11000	S	10011	S	10011	P	10000	P	10000
M	01101	T	10100	T	10100	R	10010	R	10010	R	10010
P	10000	P	10000	P	10000	P	10000	S	10011	S	10011
L	01100	R	10010	R	10010	T	10100	T	10100	T	10100
E	00101	S	10011	X	11000	X	11000	X	11000	X	11000

Figure 10.3
Binary quicksort example (key bits exposed)

We derive this figure from Figure 10.2 by translating the keys to their binary encoding, compressing the table such that the independent subfile sorts are shown as though they happen in parallel, and transposing rows and columns. The first stage splits the file into a subfile with all keys beginning with 0, and a subfile with all keys beginning with 1. Then, the first subfile is split into one subfile with all keys beginning with 00, and another with all keys beginning with 01; independently, at some other time, the other subfile is split into one subfile with all keys beginning with 10, and another with all keys beginning with 11. The process stops when the bits are exhausted (for duplicate keys, in this example) or the subfiles are of size 1.

For full-word keys consisting of random bits, the starting point in Program 10.1 should be the leftmost bit of the words, or bit 0. In general, the starting point that should be used depends in a straightforward way on the application, on the number of bits per word in the machine, and on the machine representation of integers and negative numbers. For the one-letter 5-bit keys in Figures 10.2 and 10.3, the starting point on a 32-bit machine would be bit 27.

This example highlights a potential problem with binary quicksort in practical situations: Degenerate partitions (partitions with all keys having the same value for the bit being used) can happen frequently. It is not uncommon to sort small numbers (with many leading zeros) as in our examples. The problem also occurs in keys comprising characters: for example, suppose that we make up 32-bit keys from four characters by encoding each in a standard 8-bit code and then putting them together. Then, degenerate partitions are likely to occur at the beginning of each character position, because, for example, lowercase letters all begin with the same bits in most character codes. This problem is typical of the effects that we need to address when sorting encoded data, and similar problems arise in other radix sorts.

Once a key is distinguished from all the other keys by its left bits, no further bits are examined. This property is a distinct advantage in some situations; it is a disadvantage in others. When the keys are truly random bits, only about $\lg N$ bits per key are examined, and that could be many fewer than the number of bits in the keys. This fact is discussed in Section 10.6; see also Exercise 10.5 and Figure 10.1. For example, sorting a file of 1000 records with random keys might

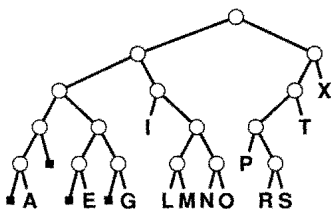


Figure 10.4
Binary quicksort partitioning trie

This tree describes the partitioning structure for binary quicksort, corresponding to Figures 10.2 and 10.3. Because no item is necessarily put into position, the keys correspond to external nodes in the tree. The structure has the following property: Following the path from the root to any key, taking 0 for left branches and 1 for right branches, gives the leading bits of the key. These are precisely the bits that distinguish the key from other keys during the sort. The small black squares represent the null partitions (when all the keys go to the other side because their leading bits are the same). This happens only near the bottom of the tree in this example, but could happen higher up in the tree: For example, if *l* or *X* were not among the keys, their node would be replaced by a null node in this drawing. Note that duplicated keys (*A* and *E*) cannot be partitioned (the sort puts them in the same subfile only after all their bits are exhausted).

involve examining only about 10 or 11 bits from each key (even if the keys are, say, 64-bit keys). On the other hand, all the bits of equal keys are examined. Radix sorting simply does not work well on files that contain huge numbers of duplicate keys that are not short. Binary quicksort and the standard method are both fast if keys to be sorted comprise truly random bits (the difference between them is primarily determined by the difference in cost between the bit-extraction and comparison operations), but the standard quicksort algorithm can adapt better to nonrandom sets of keys, and 3-way quicksort is ideal when duplicate keys predominate.

As it was with quicksort, it is convenient to describe the partitioning structure with a binary tree (as depicted in Figure 10.4): The root corresponds to a subfile to be sorted, and its two subtrees correspond to the two subfiles after partitioning. In standard quicksort, we know that at least one record is put into position by the partitioning process, so we put that key into the root node; in binary quicksort, we know that keys are in position only when we get to a subfile of size 1 or we have exhausted the bits in the keys, so we put the keys at the bottom of the tree. Such a structure is called a *binary trie*—properties of tries are covered in detail in Chapter 15. For example, one important property of interest is that the structure of the trie is completely determined by the key values, rather than by their order.

Partitioning divisions in binary quicksort depend on the binary representation of the range and number of items being sorted. For example, if the files are random permutations of the integers less than $171 = 10101011_2$, then partitioning on the first bit is equivalent to partitioning about the value 128, so the subfiles are unequal (one of size 128 and the other of size 43). The keys in Figure 10.5 are random 8-bit values, so this effect is absent there, but the effect is worthy of note now, lest it come as a surprise when we encounter it in practice.

We can improve the basic recursive implementation in Program 10.1 by removing recursion and treating small subfiles differently, just as we did for standard quicksort in Chapter 7.

Exercises

- ▷ 10.8 Draw the trie in the style of Figure 10.2 that corresponds to the partitioning process in radix quicksort for the key EASYQUESTION.

10.9 Compare the number of exchanges used by binary quicksort with the number used by the normal quicksort for the file of 3-bit binary numbers 001, 011, 101, 110, 000, 001, 010, 111, 110, 010.

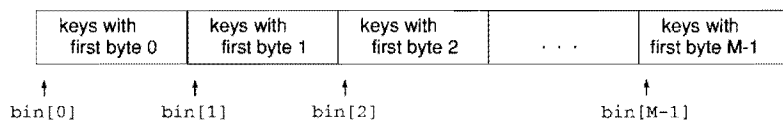
- **10.10** Why is it not as important to sort the smaller of the two subfiles first in binary quicksort as it was for normal quicksort?
- **10.11** Describe what happens on the second level of partitioning (when the left subfile is partitioned and when the right subfile is partitioned) when we use binary quicksort to sort a random permutation of the nonnegative integers less than 171.

10.12 Write a program that, in one preprocessing pass, identifies the number of leading bit positions where all keys are equal, then calls a binary quicksort that is modified to ignore those bit positions. Compare the running time of your program with that of the standard implementation for $N = 10^3$, 10^4 , 10^5 , and 10^6 when the input is 32-bit words of the following format: The rightmost 16 bits are uniformly random, and the leftmost 16 bits are all 0 except with a 1 in position i if there are i 1s in the right half.

10.13 Modify binary quicksort to check explicitly for the case that all keys are equal. Compare the running time of your program with that of the standard implementation for $N = 10^3$, 10^4 , 10^5 , and 10^6 with the input described in Exercise 10.12.

10.3 MSD Radix Sort

Using just 1 bit in radix quicksort amounts to treating keys as radix-2 (binary) numbers and considering the most significant digits first. Generalizing, suppose that we wish to sort radix- R numbers by considering the most significant bytes first. Doing so requires partitioning the array into R , rather than just two, different parts. Traditionally we refer to the partitions as *bins* or *buckets* and think of the algorithm as using a group of R bins, one for each possible value of the first digit, as indicated in the following diagram:



We pass through the keys, distributing them among the bins, then recursively sort the bin contents on keys with 1 fewer byte.

Figure 10.6 shows an example of MSD radix sorting on a random permutation of integers. By contrast with binary quicksort, this

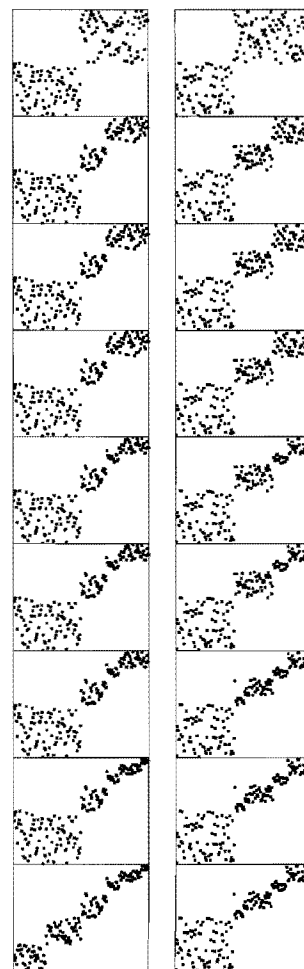


Figure 10.5
Dynamic characteristics of binary quicksort on a large file

Partitioning divisions in binary quicksort are less sensitive to key order than they are in standard quicksort. Here, two different random 8-bit files lead to virtually identical partitioning profiles.

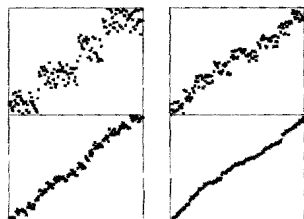


Figure 10.6
Dynamic characteristics of
MSD radix sort

Just one stage of MSD radix sort can nearly complete a sort task, as shown in this example with random 8-bit integers. The first stage of an MSD sort, on the leading 2 bits (left), divides the file into four subfiles. The next stage divides each of those into four subfiles. An MSD sort on the leading 3 bits (right) divides the file into eight subfiles, in just one distribution-counting pass. At the next level, each of those subfiles is divided into eight parts, leaving just a few elements in each.

algorithm can bring a file nearly into order rather quickly, even on the first partition, if the radix is sufficiently large.

As mentioned in Section 10.2, one of the most attractive features of radix sorting is the intuitive and direct manner in which it adapts to sorting applications where keys are strings of characters. This observation is especially true in C and other programming environments that provide direct support for processing strings. For MSD radix sorting, we simply use a radix corresponding to the byte size. To extract a digit, we load a byte; to move to the next digit, we increment a string pointer. For the moment, we consider fixed-length keys; we shall see shortly that variable-length string keys are easy to handle with the same basic mechanisms.

Figure 10.7 shows an example of MSD radix sorting on three-letter words. For simplicity, this figure assumes that the radix is 26, although in most applications we would use a larger radix corresponding to the character encodings. First, the words are partitioned so all those that start with a appear before those that start with b, and so forth. Then, the words that start with a are sorted recursively, then the words that start with b are sorted, and so forth. As is obvious from the example, most of the work in the sort lies in partitioning on the first letter; the subfiles that result from the first partition are small.

As we saw for quicksort in Chapter 7 and Section 10.2 and for mergesort in Chapter 8, we can improve the performance of most recursive programs by using a simple algorithm for small cases. Using a different method for small subfiles (bins containing a small number of elements) is essential for radix sorting, because there are so many of them! Moreover, we can tune the algorithm by adjusting the value of R because there is a clear tradeoff: If R is too large, the cost of initializing and checking the bins dominates; if it is too small, the method does not take advantage of the potential gain available by subdividing into as many pieces as possible. We return to these issues at the end of this section and in Section 10.6.

To implement MSD radix sort, we need to generalize the methods for partitioning an array that we studied in relation to quicksort implementations in Chapter 7. These methods, which are based on pointers that start from the two ends of the array and meet in the middle, work well when there are just two or three partitions, but do not immediately generalize. Fortunately, the *key-indexed counting* method

from Chapter 6 for sorting files with key values in a small range suits our needs perfectly. We use a table of counts and an auxiliary array; on a first pass through the array, we count the number of occurrences of each leading digit value. These counts tell us where the partitions will fall. Then, on a second pass through the array, we use the counts to move items to the appropriate position in the auxiliary array.

Program 10.2 implements this process. Its recursive structure generalizes quicksort's, so the same issues that we considered in Section 7.3 need to be addressed. Should we do the largest of the subfiles last to avoid excessive recursion depth? Probably not, because the recursion depth is limited by the length of the keys. Should we sort small subfiles with a simple method such as insertion sort? Certainly, because there are huge numbers of them.

To do the partitioning, Program 10.2 uses an auxiliary array of size equal to the size of the array to be sorted. Alternatively, we could choose to use in-place key-indexed counting (see Exercises 10.17 and 10.18). We need to pay particular attention to space, because the recursive calls might use excessive space for local variables. In Program 10.2, the temporary buffer for moving keys (*aux*) can be global, but the array that holds the counts and the partition positions (*count*) must be local.

Extra space for the auxiliary array is not a major concern in many practical applications of radix sorting that involve long keys and records, because a pointer sort should be used for such data. Therefore, the extra space is for rearranging pointers, and is small compared to the space for the keys and records themselves (although still not insignificant). If space is available and speed is of the essence (a common situation when we use radix sorts), we can also eliminate the time required for the array copy by recursive argument switchery, in the same manner as we did for mergesort in Section 10.4.

For random keys, the number of keys in each bin (the size of the subfiles) after the first pass will be N/R on the average. In practice, the keys may not be random (for example, when the keys are strings representing English-language words, we know that few start with *x* and none start with *xx*), so many bins will be empty and some of the nonempty ones will have many more keys than others do (see Figure 10.8). Despite this effect, the multiway partitioning process

now	ace	ace	ace
for	ago	ago	ago
tip	and	and	and
ilk	bet	bet	bet
dim	cab	cab	cab
tag	caw	caw	caw
jot	cue	cue	cue
sob	dim	dim	dim
nob	dug	dug	dug
sky	egg	egg	egg
hut	for	few	fee
ace	fee	fee	few
bet	few	for	for
men	gig	gig	gig
egg	hut	hut	hut
few	ilk	ilk	ilk
jay	jam	jay	jam
owl	jay	jam	jay
joy	jot	jot	jot
rap	joy	joy	joy
gig	men	men	men
wee	now	now	nob
was	nob	nob	now
cab	owl	owl	owl
wad	rap	rap	rap
caw	sob	sky	sky
cue	sky	sob	sob
fee	tip	tag	tag
tap	tag	tap	tap
ago	tap	tar	tar
tar	tar	tip	tip
jam	wee	wad	wad
dug	was	was	was
and	wad	wee	wee

Figure 10.7
MSD radix sort example

We divide the words into 26 bins according to the first letter. Then, we sort all the bins by the same method, starting at the second letter.

Program 10.2 MSD radix sort

We derive this program from Program 8.17 (key-indexed-counting sort) by changing key references to key-digit references, and adding a loop at the end that does recursive calls for each subfile of keys starting with the same digit. For variable-length keys terminated by 0 digits (such as C strings), omit the first if statement and the first recursive call. This implementation uses an auxiliary array (aux) that is big enough to hold a copy of the input.

```
#define bin(A) 1+count[A]
void radixMSD(Item a[], int l, int r, int w)
{ int i, j, count[R+1];
  if (w > bytesword) return;
  if (r-l <= M) { insertion(a, l, r); return; }
  for (j = 0; j < R; j++) count[j] = 0;
  for (i = l; i <= r; i++)
    count[digit(a[i], w) + 1]++;
  for (j = 1; j < R; j++)
    count[j] += count[j-1];
  for (i = l; i <= r; i++)
    aux[count[digit(a[i], w)]++] = a[i];
  for (i = l; i <= r; i++) a[i] = aux[i-l];
  radixMSD(a, l, bin(0)-1, w+1);
  for (j = 0; j < R-1; j++)
    radixMSD(a, bin(j), bin(j+1)-1, w+1);
}
```

will generally be effective in dividing a large file to be sorted into many smaller ones.

Another natural way to implement MSD radix sorting is to use linked lists. We keep one linked list for each bin: On a first pass through the items to be sorted, we insert each item into the appropriate linked list, according to its leading digit value. Then, we sort the sublists, and stitch together all the linked lists to make a sorted whole. This approach presents a challenging programming exercise (see Exercise 10.36). Stitching together the lists requires keeping track of the beginning and the end of all the lists, and, of course, many of the lists are likely to be empty.

To achieve good performance using radix sort for a particular application, we need to limit the number of empty bins encountered by choosing appropriate values both for the radix size and for the cutoff for small subfiles. As a concrete example, suppose that 2^{24} (about sixteen million) 64-bit integers are to be sorted. To keep the table of counts small by comparison with the file size, we might choose a radix of $R = 2^{16}$, corresponding to checking 16 bits of the keys. But after the first partition, the average file size is only 2^8 , and a radix of 2^{16} for such small files is overkill. To make matters worse, there can be huge numbers of such files: about 2^{16} of them in this case. For each of those 2^{16} files, the sort sets 2^{16} counters to zero, then checks that all but about 2^8 of them are nonzero, and so forth, for a cost of *at least* 2^{32} arithmetic operations. Program 10.2, which is implemented on the assumption that most bins are nonempty, does more than a few arithmetic operations for each empty bin (for example, it does recursive calls for all the empty bins), so its running time would be huge for this example. A more appropriate radix for the second level might be 2^8 or 2^4 . In short, we should be certain not to use large radices for small files in a MSD radix sort. We shall consider this point in detail in Section 10.6, when we look carefully at the performance of the various methods.

If we set $R = 256$ and eliminate the recursive call for bin 0, then Program 10.2 is an effective way to sort C strings. If we know that the lengths of all the strings are less than a certain fixed length, we can set the variable `bytesword` to that length, or we can eliminate the test on `bytesword` to sort standard variable-length character strings. For sorting strings, we normally would implement the `digit` abstract operation as a single array reference, as we discussed in Section 10.1. By adjusting R and `bytesword` (and testing their values), we can easily modify Program 10.2 to handle strings from nonstandard alphabets or in nonstandard formats involving length restrictions or other conventions.

String sorting again illustrates the importance of managing empty bins properly. Figure 10.8 shows the partitioning process for an example like Figure 10.7, but with two-letter words and with the empty bins shown explicitly. In this example, we radix sort two-letter words using radix 26, so there are 26 bins at every stage. In the first stage,

```

no an am
if am an
be at as
do as at

he be be

an by by
by do do
of go go
us he he

on if if

am is in
we it is
is in it

at me me
it no no

to of of

or on on
me or or

go to to
in us us
as we we

```

Figure 10.8
MSD radix sort example
(with empty bins)

Excessive numbers of empty bins are encountered, even in the second stage, for small files.

there are not many empty bins; in the second stage, however, most bins are empty.

An MSD radix-sorting function divides the file on the first digit of the keys, then recursively calls itself for subfiles corresponding to each value. Figure 10.9 shows this recursive-call structure for MSD radix sorting for the example in Figure 10.8. The call structure corresponds to a *multiway trie*, a direct generalization of the trie structure for binary quicksort in Figure 10.4. Each node corresponds to a recursive call on the MSD sort for some subfile. For example, the subtree of the root with root labeled *o* corresponds to sorting the subfile consisting of the three keys *of*, *on*, and *or*.

These figures make obvious the presence of significant numbers of empty bins in MSD sorting with strings. In Section 10.4, we study one way to cope with this problem; in Chapter 15, we examine explicit uses of trie structures in string-processing applications. Generally, we work with compact representations of the trie structures that do not include the nodes corresponding to the empty bins and that have the labels moved from the edges to the nodes below, as illustrated in Figure 10.10, the structure that corresponds to the recursive call structure (ignoring empty bins) for the three-letter MSD radix-sorting example of Figure 10.7. For example, the subtree of the root with root labeled *j* corresponds to sorting the bin containing the four keys *jam*, *jay*, *jot*, and *joy*. We examine properties of such tries in detail in Chapter 15.

The main challenge in getting maximum efficiency in a practical MSD radix sort for keys that are long strings is to deal with lack of randomness in the data. Typically, keys may have long stretches of equal or unnecessary data, or parts of them might fall in only a narrow range. For example, an information-processing application for student data records might have keys with fields corresponding to graduation year (4 bytes, but one of four different values), state names (perhaps 10 bytes, but one of 50 different values), and gender (1 byte with one of two given values), as well as to a person's name (more similar to random strings, but probably not short, with nonuniform letter distributions, and with trailing blanks in a fixed-length field). All these various restrictions lead to large numbers of empty bins during the MSD radix sort (see Exercise 10.23).

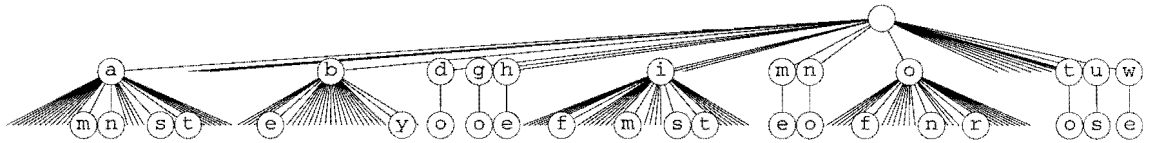


Figure 10.9
Recursive structure of MSD
radix sort.

This tree corresponds to the operation of the recursive MSD radix sort in Program 10.2 on the two-letter MSD sorting example in Figure 10.8. If the file size is 1 or 0, there are no recursive calls. Otherwise, there are 26 calls: one for each possible value of the current byte.

One practical way to cope with this problem is to develop a more complex implementation of the abstract operation of accessing bytes that takes into account any specialized knowledge that we might have about the strings being sorted. Another method that is easy to implement, which is called the *bin-span heuristic*, is to keep track of the high and low ends of the range of nonempty bins during the counting phase, then to use only bins in that range (perhaps also including special cases for a few special key values, such as 0 or blank). This arrangement is attractive for the kind of situation described in the previous paragraph. For example, with radix-256 alphanumeric data, we might be working with numbers in one section of the keys and thus have only 10 nonempty bins corresponding to the digits, while we might be working with uppercase letters in another section of the keys and thus have only 26 nonempty bins corresponding to them.

There are various alternatives that we might try for extending the bin-span heuristic (*see reference section*). For example, we could consider keeping track of the nonempty bins in an auxiliary data structure, and only keep counters and do the recursive calls for those. Doing so (and even the bin-span heuristic itself) is probably overkill for this situation, however, because the cost savings is negligible unless the radix is huge or the file size is tiny, in which case we should be using a smaller radix or sorting the file with some other method. We might achieve some of the same cost savings that we could achieve by adjusting the radix or switching to a different method for small files by using an ad hoc method, but we could not do so as easily. In Section 10.4, we shall consider yet another version of quicksort that does handle the empty-bin problem gracefully.

Exercises

- ▷ 10.14 Draw the compact trie structure (with no empty bins and with keys in nodes, as in Figure 10.10) corresponding to Figure 10.9.
- ▷ 10.15 How many nodes are there in the full trie corresponding to Figure 10.10?

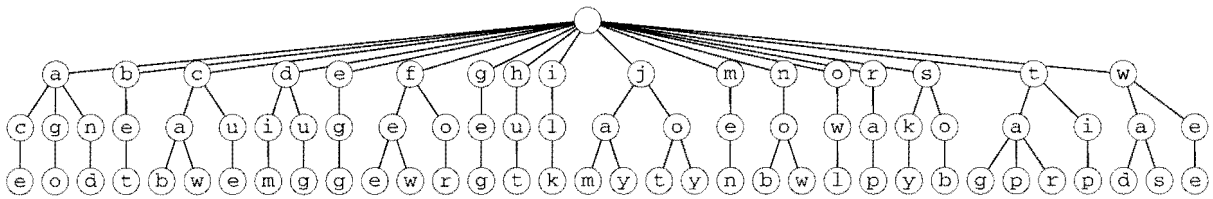


Figure 10.10
Recursive structure of
MSD radix sort (null sub-
files ignored)

This representation of the recursive structure of MSD radix sort is more compact than the one in Figure 10.9. Each node in this tree is labeled with the value of the $(i - 1)$ st digit of certain keys, where i is the distance from the node to the root. Each path from the root to the bottom of the tree corresponds to a key; putting the node labels together gives the key. This tree corresponds to the three-letter MSD sorting example in Figure 10.7.

▷ **10.16** Show how the set of keys now is the time for all good people to come the aid of their party is partitioned with MSD radix sort.

• **10.17** Write a program that does four-way partitioning in place, by counting the frequency of occurrence of each key as in key-indexed counting, then using a method like Program 6.14 to move the keys.

•• **10.18** Write a program to solve the general R -way partitioning problem, using the method sketched in Exercise 10.17.

10.19 Write a program that generates random 80-byte keys. Use this key generator to generate N random keys, then sort them with MSD radix sort, for $N = 10^3, 10^4, 10^5$, and 10^6 . Instrument your program to print out the total number of key bytes examined for each sort.

○ **10.20** What is the rightmost key byte position that you would expect the program in Exercise 10.19 to access for each of the given values of N ? If you have done that exercise, instrument your program to keep track of this quantity, and compare your theoretical result with empirical results.

10.21 Write a key generator that generates keys by shuffling a random 80-byte sequence. Use your key generator to generate N random keys, then sort them with MSD radix sort, for $N = 10^3, 10^4, 10^5$, and 10^6 . Compare your performance results with those for the random case (see Exercise 10.19).

10.22 What is the rightmost key byte position that you would expect the program in Exercise 10.21 to access for each value of N ? If you have done that exercise, compare your theoretical result with empirical results from your program.

10.23 Write a key generator that generates 30-byte random strings made up of three fields: a four-byte field with one of a set of 10 given strings; a 10-byte field with one of a set of 50 given strings; a 1-byte field with one of two given values; and a 15-byte field with random left-justified strings of letters equally likely to be four through 15 characters long. Use your key generator to generate N random keys, then sort them with MSD radix sort, for $N = 10^3, 10^4, 10^5$, and 10^6 . Instrument your program to print out the total number of key bytes examined. Compare your performance results with those for the random case (see Exercise 10.19).

10.24 Modify Program 10.2 to implement the bin-span heuristic. Test your program on the data of Exercise 10.23.

10.4 Three-Way Radix Quicksort

Another way to adapt quicksort for MSD radix sorting is to use three-way partitioning on the leading byte of the keys, moving to the next byte on only the middle subfile (keys with leading byte equal to that of the partitioning element). This method is easy to implement (the one-sentence description plus the three-way partitioning code in Program 7.5 suffices, essentially), and it adapts well to a variety of situations. Program 10.3 is a full implementation of this method.

In essence, doing three-way radix quicksort amounts to sorting the file on the leading characters of the keys (using quicksort), then applying the method recursively on the remainder of the keys. For sorting strings, the method compares favorably with normal quicksort and with MSD radix sort. Indeed, it might be viewed as a hybrid of these two algorithms.

To compare three-way radix quicksort to standard MSD radix sort, we note that it divides the file into only three parts, so it does not get the benefit of the quick multiway partition, especially in the early stages of the sort. On the other hand, for later stages, MSD radix sort involves large numbers of empty bins, whereas three-way radix quicksort adapts well to handle duplicate keys, keys that fall into a small range, small files, and other situations where MSD radix sort might run slowly. Of particular importance is that the partitioning adapts to different types of nonrandomness in different parts of the key. Furthermore, no auxiliary array is required. Balanced against all these advantages is that extra exchanges are required to get the effect of the multiway partition via a sequence of three-way partitions when the number of subfiles is large.

Figure 10.11 shows an example of the operation of this method on the three-letter-word sorting problem of Figure 10.7. Figure 10.13 depicts the recursive-call structure. Each node corresponds to precisely three recursive calls: for keys with a smaller first byte (left child), for keys with first byte equal (middle child), and for keys with first byte larger (right child).

When the sort keys fit the abstraction of Section 10.2, standard quicksort (and all the other sorts in Chapters 6 through 9) can be viewed as an MSD radix sort, because the compare function has to access the most significant part of the key first (see Exercise 10.3).

```

now gig ace  ago  ago
for for bet  bet  ace
tip dug dug  and  and
ilk ilk cab  ace  bet
dim dim dim  cab
tag ago ago  caw
jot and and  cue
sob fee egg  egg
nob cue cue  dug
sky caw caw  dim
hut hut fee
ace ace for
bet bet few
men cab ilk
egg egg gig
few few hut
jay jay jam
owl jot jay
joy joy joy
rap jam jot
gig owl owl men
wee wee now owl
was was nob nob
cab men men now
wad wad rap
caw sky sky sky sky
cue nob was tip sob
fee sob sob sob tip tar
tap tap tap tap tap tap
ago tag tag tag tag tag
tar tar tar tar tar tip
dug tip tip was
and now wee wee
jam rap wad wad

```

Figure 10.11
Three-way radix quicksort

We divide the file into three parts: words beginning with a through i, words beginning with j, and words beginning with k through z. Then, we sort recursively.

Program 10.3 Three-way radix quicksort

This MSD radix sort is essentially the same code as quicksort with three-way partitioning (Program 9.5), but with the following changes: (i) key references become key-byte references, (ii) the current byte is added as a parameter to the recursive routine, and (iii) the recursive calls for the middle subfile move to the next byte. We avoid moving past the ends of strings by checking whether the partitioning value is 0 before recursive calls that move to the next byte. When the partitioning value is 0, the left subfile is empty, the middle subfile corresponds to the keys that the program has found to be equal, and the right subfile corresponds to longer strings that need to be processed further.

```
#define ch(A) digit(A, D)
void quicksortX(Item a[], int l, int r, int D)
{
    int i, j, k, p, q; int v;
    if (r-l <= M) { insertion(a, l, r); return; }
    v = ch(a[r]); i = l-1; j = r; p = l-1; q = r;
    while (i < j)
    {
        while (ch(a[++i]) < v) ;
        while (v < ch(a[--j])) if (j == l) break;
        if (i > j) break;
        exch(a[i], a[j]);
        if (ch(a[i])==v) { p++; exch(a[p], a[i]); }
        if (v==ch(a[j])) { q--; exch(a[j], a[q]); }
    }
    if (p == q)
        { if (v != '\0') quicksortX(a, l, r, D+1);
          return; }
    if (ch(a[i]) < v) i++;
    for (k = l; k <= p; k++, j--) exch(a[k], a[j]);
    for (k = r; k >= q; k--, i++) exch(a[k], a[i]);
    quicksortX(a, l, j, D);
    if ((i == r) && (ch(a[i]) == v)) i++;
    if (v != '\0') quicksortX(a, j+1, i-1, D+1);
    quicksortX(a, i, r, D);
}
```

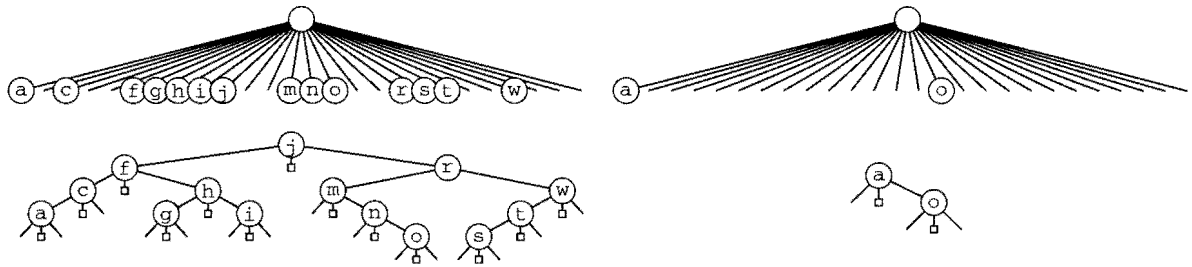


Figure 10.12
Example of trie nodes for
three-way radix quicksort

Three-way radix quicksort addresses the empty-bin problem for MSD radix sort by doing three-way partitioning to eliminate 1 byte value and (recursively) to work on the others. This action corresponds to replacing each M -way node in the trie that describes the recursive call structure of MSD radix sort (see Figure 10.9) by a ternary tree with an internal node for each nonempty bin. For full nodes (left), this change costs time without saving much space, but for empty nodes (right), the time cost is minimal and the space savings is considerable.

For example, if the keys are strings, the compare function should access only the leading bytes if they are different, the leading 2 bytes if the first bytes are the same and the second different, and so forth. The standard algorithm thus automatically realizes some of the same performance gain that we seek in MSD radix sorting (see Section 7.7). The essential difference is that the standard algorithm cannot take special action when the leading bytes are equal. Indeed, one way to think of Program 10.3 is as a way for quicksort to keep track of what it knows about leading digits of items after they have been involved in multiple partitions. In the small subfiles, where most of the comparisons in the sort are done, the keys are likely to have many equal leading bytes. The standard algorithm has to scan over all those bytes for each comparison; the three-way algorithm avoids doing so.

Consider a case where the keys are long (and are fixed length, for simplicity), but most of the leading bytes are all equal. In such a situation, the running time of normal quicksort would be proportional to the word length *times* $2N \ln N$, whereas the running time of the radix version would be proportional to N times the word length (to discover all the leading equal bytes) *plus* $2N \ln N$ (to do the sort on the remaining short keys). That is, this method could be up to a factor of $\ln N$ faster than normal quicksort, counting just the cost of comparisons. It is not unusual for keys in practical sorting applications to have characteristics similar to this artificial example (see Exercise 10.25).

Another interesting property of three-way radix quicksort is that it has no direct dependencies on the size of the radix. For other radix sorting methods, we have to maintain an auxiliary array indexed by radix value, and we need to ensure that the size of this array is not appreciably larger than the file size. For this method, there is no such

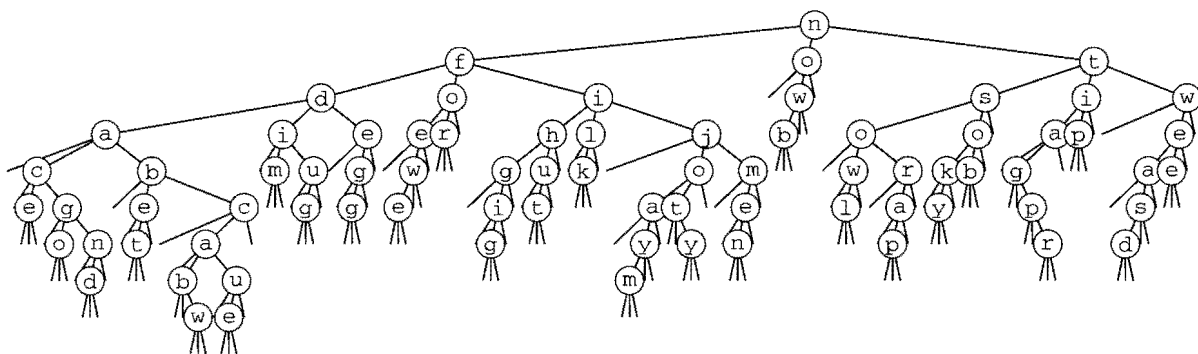


Figure 10.13
Recursive structure of three-way radix quicksort

This tree-trie combination corresponds to a substitution of the 26-way nodes in the trie in Figure 10.10 by ternary binary search trees, as illustrated in Figure 10.12. Any path from the root to the bottom of the tree that ends in a middle link defines a key in the file, given by the characters in the nodes left by middle links in the path. Figure 10.10 has 1035 null links that are not depicted; all the 155 null links in this tree are shown here. Each null link corresponds to an empty bin, so this difference illustrates how three-way partitioning can cut dramatically the number of empty bins encountered in MSD radix sorting.

table. Taking the radix to be extremely large (larger than the word size) reduces the method to normal quicksort, and taking the radix to be 2 reduces it to binary quicksort, but intermediate values of the radix give us an efficient way to deal with equal stretches among pieces of keys.

For many practical applications, we can develop a hybrid method with excellent performance by using standard MSD radix sort for large files, to get the advantage of multiway partitioning, and a three-way radix quicksort with a smaller radix for smaller files, to avoid the negative effects of large numbers of empty bins.

Three-way radix quicksort is also applicable when the keys to be sorted are *vectors*. That is, if the keys are made up of independent components (each an abstract key), we might wish to reorder records such that they are in order according to the first components of the keys, *and* in order according to the second component of the keys if the first components are equal, and so forth. We can think of vector sorting as a generalization of radix sorting where we take R to be arbitrarily large. When we adapt Program 10.3 to this application, we refer to it as *multikey quicksort*.

Exercises

10.25 For $d > 4$, suppose that keys consist of d bytes, with the final 4 bytes having random values and all the other bytes having value 0. Estimate the number of bytes examined when you sort the file using three-way radix quicksort (Program 10.3) and normal quicksort (Program 7.1) for files of size N for large N , and calculate the ratio of the running times.

10.26 Empirically determine the byte size for which three-way radix quicksort runs fastest, for random 64-bit keys with $N = 10^3, 10^4, 10^5$, and 10^6 .

- **10.27** Develop an implementation of three-way radix quicksort for linked lists.

10.28 Develop an implementation of multikey quicksort for the case where the keys are vectors of t floating point numbers, using equality testing among floating point numbers as described in Exercise 4.1.

10.29 Using the key generator of Exercise 10.19, run three-way radix quicksort for $N = 10^3, 10^4, 10^5$, and 10^6 . Compare its performance with that of MSD radix sort.

10.30 Using the key generator of Exercise 10.21, run three-way radix quicksort for $N = 10^3, 10^4, 10^5$, and 10^6 . Compare its performance with that of MSD radix sort.

10.31 Using the key generator of Exercise 10.23, run three-way radix quicksort for $N = 10^3, 10^4, 10^5$, and 10^6 . Compare its performance with that of MSD radix sort.

10.5 LSD Radix Sort

An alternative radix-sorting method is to examine the bytes from right to left. Figure 10.14 shows how our three-letter word sorting task is accomplished in just three passes through the file. We sort the file according to the final letter (using key-indexed counting), then according to the middle letter, then according to the first letter.

It is not easy, at first, to be convinced that the method works; in fact, it does not work at all unless the sort method used is *stable* (see Definition 6.1). Once stability has been identified as being significant, a simple proof that LSD radix sorting works is easy to articulate: After putting keys into order on their i trailing bytes (in a stable manner), we know that any two keys appear in proper order (on the basis of the bits so far examined) in the file either because the first of their i trailing bytes are different, in which case the sort on that byte put them in the proper order, or because the first of their i th trailing bytes are the same, in which case they are in proper order because of stability. Stated another way, if the $w - i$ bytes that have not been examined for a pair of keys are identical, any difference between the keys is restricted to the i bytes already examined, and the keys have been properly ordered, and will remain so because of stability. If, on the other hand, the $w - i$ bytes that have not been examined are different, the i bytes already examined do not matter, and a later pass will correctly order the pair based on the more-significant differences.

now	sob	cab	ace
for	nob	wad	ago
tip	cab	tag	and
ilk	wad	jam	bet
dim	and	rap	cab
tag	ace	tap	caw
jot	wee	tar	cue
sob	cue	was	dim
nob	fee	caw	dug
sky	tag	raw	egg
hut	egg	jay	fee
ace	gig	ace	few
bet	dug	wee	for
men	ilk	fee	gig
egg	owl	men	hut
few	dim	bet	ilk
jay	jam	few	jam
owl	men	egg	jay
joy	ago	ago	jot
rap	tip	gig	joy
gig	rap	dim	men
wee	tap	tip	nob
was	for	sky	now
cab	tar	ilk	owl
wad	was	and	rap
tap	jot	sob	raw
caw	hut	nob	sky
cue	bet	for	sob
fee	you	jot	tag
raw	now	you	tap
ago	few	now	tar
tar	caw	joy	tip
jam	raw	cue	wad
dug	sky	dug	was
you	jay	hut	wee
and	joy	owl	you

Figure 10.14
LSD radix sort example

Three-letter words are sorted in three passes (left to right) with LSD radix sorting.

Program 10.4 LSD radix sort

This program implements key-indexed counting on the bytes in the words, moving right to left. The key-indexed counting implementation must be stable. If R is 2 (and therefore `bytesword` and `bitsword` are the same), this program is *straight radix sort*—a right-to-left bit-by-bit radix sort (see Figure 10.15).

```
void radixLSD(Item a[], int l, int r)
{
    int i, j, w, count[R+1];
    for (w = bytesword-1; w >= 0; w--)
    {
        for (j = 0; j < R; j++) count[j] = 0;
        for (i = l; i <= r; i++)
            count[digit(a[i], w) + 1]++;
        for (j = 1; j < R; j++)
            count[j] += count[j-1];
        for (i = l; i <= r; i++)
            aux[count[digit(a[i], w)]+1] = a[i];
        for (i = l; i <= r; i++) a[i] = aux[i-l];
    }
}
```

The stability requirement means, for example, that the partitioning method used for binary quicksort could not be used for a binary version of this right-to-left sort. On the other hand, key-indexed counting *is* stable, and immediately leads to a classic and efficient algorithm. Program 10.4 is an implementation of this method. An auxiliary array for the distribution seems to be required—the technique of Exercises 10.17 and 10.18 for doing the distribution in place sacrifices stability to avoid using the auxiliary array.

LSD radix sorting is the method used by old computer-card-sorting machines. Such machines had the capability of distributing a deck of cards among 10 bins, according to the pattern of holes punched in the selected columns. If a deck of cards had numbers punched in a particular set of columns, an operator could sort the cards by running them through the machine on the rightmost digit, then picking up and stacking the output decks in order, then running

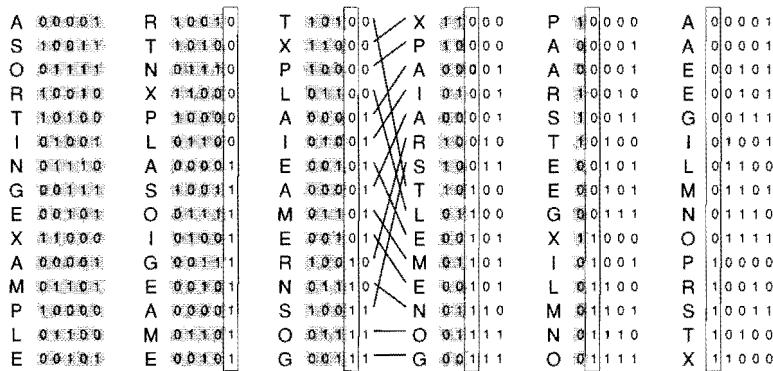


Figure 10.15
LSD (binary) radix sort exam-
ple (key bits exposed)

This diagram depicts a right-to-left bit-by-bit radix sort working on our file of sample keys. We compute the i th column from the $(i - 1)$ st column by extracting (in a stable manner) all the keys with a 0 in the i th bit, then all the keys with a 1 in the i th bit. If the $(i - 1)$ st column is in order on the trailing $(i - 1)$ bits of the keys before the operation, then the i th column is in order on the trailing i bits of the keys after the operation. The movement of the keys in the third stage is indicated explicitly.

them through the machine on the next-to-rightmost digit, and so forth, until getting to the first digit. The physical stacking of the cards is a stable process, which is mimicked by key-indexed counting sort. Not only was this version of LSD radix sorting important in commercial applications in the 1950s and 1960s, but it was also used by many cautious programmers, who would punch sequence numbers in the final few columns of a program deck so as to be able to put the deck back in order mechanically if it were accidentally dropped.

Figure 10.15 depicts the operation of binary LSD radix sort on our sample keys, for comparison with Figure 10.3. For these 5-bit keys, the sort is completed in five passes, moving right to left through the keys. Sorting records with single-bit keys amounts to partitioning the file such that all the records with 0 keys appear before all the records with 1 keys. As just mentioned, we cannot use the partitioning strategy that we discussed at the beginning of this chapter in Program 10.1, even though it seems to solve this same problem, because it is not stable. It is worthwhile to look at radix-2 sorting, because it is often appropriate for high-performance machines and special-purpose hardware (see Exercise 10.38). In software, we use as many bits as we can to reduce the number of passes, limited only by the size of the array for the counts (see Figure 10.16).

It is typically difficult to apply the LSD approach to a string-sorting application because of variable-length keys. For MSD sorting, it is simple enough to distinguish keys according to their leading bytes, but LSD sorting is based on a fixed-length key, with the leading keys getting involved for only the final pass. Even for (long) fixed-length

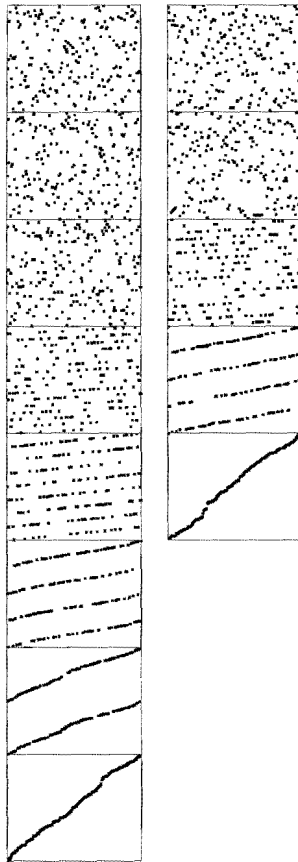


Figure 10.16
Dynamic characteristics of
LSD radix sort

This diagram shows the stages of LSD radix sort on random 8-bit keys, for both radix 2 (left) and radix 4, which comprises every other stage from the radix-2 diagram (right). For example, when two bits remain (second-to-last stage on the left, next-to-last stage on the right), the file consists of four intermixed sorted subfiles consisting of the keys beginning with 00, 01, 10, and 11.

keys, LSD radix sorting would seem to be doing unnecessary work on the right parts of the keys, since, as we have seen, only the left parts of the keys are typically used in the sort. We shall see a way to address this problem in Section 10.7, after we have examined the properties of radix sorts in detail.

Exercises

10.32 Using the key generator of Exercise 10.19, run LSD radix sort for $N = 10^3$, 10^4 , 10^5 , and 10^6 . Compare its performance with that of MSD radix sort.

10.33 Using the key generators of Exercises 10.21 and 10.23, run LSD radix sort for $N = 10^3$, 10^4 , 10^5 , and 10^6 . Compare its performance with that of MSD radix sort.

10.34 Show the (unsorted) result of trying to use an LSD radix sort based on the binary quicksort partitioning method for the example of Figure 10.15.

▷ **10.35** Show the result of using LSD radix sort on the leading two characters for the set of keys now is the time for all good people to come the aid of their party.

- **10.36** Develop an implementation of LSD radix sort using linked lists.
- **10.37** Find an efficient method that (i) rearranges the records of a file such that all those whose keys begin with a 0 bit come before all those whose keys begin with a 1 bit, (ii) uses extra space proportional to the square root of the number of records (or less), and (iii) is stable.
- **10.38** Implement a routine that sorts an array of 32-bit words using only the following abstract operation: Given a bit position i and a pointer into the array $a[k]$, rearrange $a[k]$, $a[k+1]$, $\dots$, $a[k+63]$ in a stable manner such that those words with a 0 bit in position i appear before those words with a 1 bit in position i .

10.6 Performance Characteristics of Radix Sorts

The running time of LSD radix sort for sorting N records with w -byte keys is proportional to Nw , because the algorithm makes w passes over all N keys. This analysis does not depend on the input, as illustrated in Figure 10.17.

For long keys and short bytes, this running time is comparable to $N \lg N$: For example, if we are using a binary LSD radix sort to sort 1 billion 32-bit keys, then w and $\lg N$ are both about 32. For shorter keys and longer bytes this running time is comparable to N :

For example, if a 16-bit radix is used on 64-bit keys, then w will be 4, a small constant.

To compare properly the performance of radix sort with the performance of comparison-based algorithms, we need to account carefully for the bytes in the keys, rather than for only the number of keys.

Property 10.1 *The worst case for radix sorting is to examine all the bytes in all the keys.*

In other words, the radix sorts are *linear* in the sense that the time taken is at most proportional to the number of digits in the input. This observation follows directly from examination of the programs: No digit is examined more than once. This worst case is achieved, for all the programs we have examined, when all the keys are equal. ■

As we have seen, for random keys and for many other situations, the running time of MSD radix sorting can be *sublinear* in the total number of data bits, because the whole key does not necessarily have to be examined. The following classical result holds for arbitrarily long keys:

Property 10.2 *Binary quicksort examines about $N \lg N$ bits, on average, when sorting keys composed of random bits.*

If the file size is a power of 2 and the bits are random, then we expect one-half of the leading bits to be 0 and one-half to be 1, so the recurrence $C_N = 2C_{N/2} + N$ should describe the performance, as we argued for quicksort in Chapter 7. Again, this description of the situation is not entirely accurate, because the partition falls in the center only on the average (and because the number of bits in the keys is finite). However, the partition is much more likely to be near the center for binary quicksort than for standard quicksort, so the leading term of the running time is the same as it would be were the partitions perfect. The detailed analysis that proves this result is a classical example in the analysis of algorithms, first done by Knuth before 1973 (see *reference section*). ■

This result generalizes to apply to MSD radix sort. However, since our interest is generally in the total running time, rather than in only the key characters examined, we have to exercise caution, because part of

the running time of MSD radix sort is proportional to the size of the radix R and has nothing to do with the keys.

Property 10.3 *MSD radix sort with radix R on a file of size N requires at least $2N + 2R$ steps.*

MSD radix sort involves at least one key-indexed counting pass, and key-indexed counting consists of at least two passes through the records (one for counting and one for distributing), accounting for at least $2N$ steps, and two passes through the counters (one to initialize them to 0 at the beginning and one to determine where the subfiles are at the end), accounting for at least $2R$ steps. ■

This property almost seems too obvious to state, but it is essential to our understanding of MSD radix sort. In particular, it tells us that we cannot conclude that the running time will be low from the fact that N is small, because R could be much larger than N . In short, *some other method should be used for small files*. This observation is a solution to the empty-bins problem that we discussed at the end of Section 10.3. For example, if R is 256 and N is 2, MSD radix sort will be up to 128 times slower than the simpler method of just comparing elements. The recursive structure of MSD radix sort ensures that the recursive program will call itself for large numbers of small files. Therefore, ignoring the empty-bins problem could make the whole radix sort up to 128 times slower than it could be for this example. For intermediate situations (for example, suppose that R is 256 and N is 64), the cost is not so catastrophic, but is still significant. Using insertion sort is not wise, because its expected cost of $N^2/4$ comparisons is too high; ignoring the empty bins is not wise, because there are significant numbers of them. The simplest way to cope with this problem is to use a radix that is less than the file size.

Property 10.4 *If the radix is always less than the file size, the number of steps taken by MSD radix sort is within a small constant factor of $N \log_R N$ on the average (for keys comprising random bytes), and within a small constant factor of the number of bytes in the keys in the worst case.*

The worst-case result follows directly from the preceding discussion, and the analysis cited for Property 10.2 generalizes to give the average-case result. For large R , the factor $\log_R N$ is small, so the total time

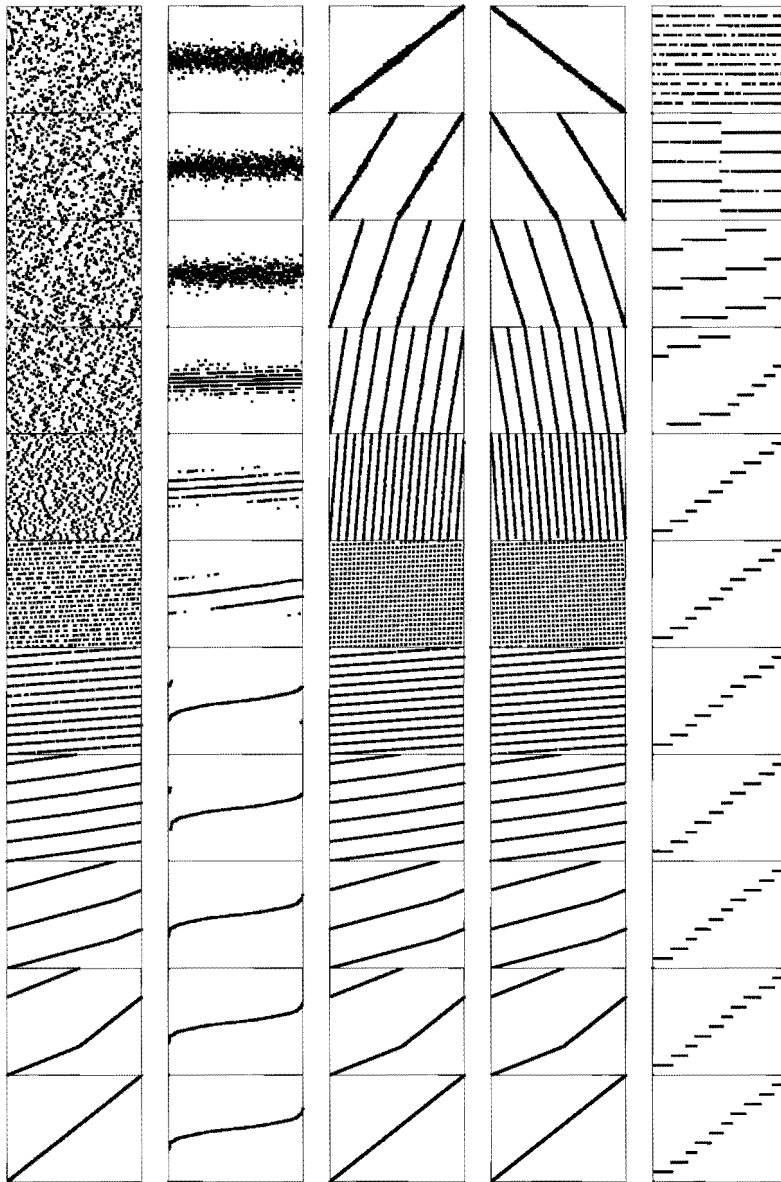


Figure 10.17
Dynamic characteristics of
LSD radix sort on vari-
ous types of files

These diagrams illustrate the stages of LSD radix sort for files of size 700 that are random, Gaussian, nearly ordered, nearly reverse ordered, and randomly ordered with 10 distinct key values (left to right). The running time is insensitive to the initial order of the input. The three files that contain the same set of keys (the first, third, and fourth all are a permutation of the integers from 1 to 700) have similar characteristics near the end of the sort.

is proportional to N for practical purposes. For example, if $R = 2^{16}$, then $\log_R N$ is less than 3 for all $N < 2^{48}$, which value certainly encompasses all practical file sizes. ■

As we do from Property 10.2 we have from Property 10.4 the important practical implication that MSD radix sorting is actually a *sublinear* function of the total number of bits for random keys that are not short. For example, sorting 1 million 64-bit random keys will require examining only the leading 20 to 30 bits of the keys, or less than one-half of the data.

Property 10.5 *Three-way radix quicksort uses $2N \ln N$ byte comparisons, on the average, to sort N (arbitrarily long) keys.*

There are two instructive ways to understand this result. First, considering the method to be equivalent to quicksort partitioning on the leading byte, then (recursively) using the same method on the subfiles, we should not be surprised that the total number of operations is about the same as for normal quicksort—but they are single-byte comparisons, not full-key comparisons. Second, considering the method from the point of view depicted in Figure 10.12, we expect that the $N \log_R N$ running time from Property 10.3 should be multiplied by a factor of $2 \ln R$ because it takes quicksort $2R \ln R$ steps to sort R bytes, as opposed to the R steps for the same bytes in the trie. We omit the full proof (*see reference section*). ■

Property 10.6 *LSD radix sort can sort N records with w -bit keys in $w/\lg R$ passes, using extra space for R counters (and a buffer for rearranging the file).*

Proof of this fact is straightforward from the implementation. In particular, if we take $R = 2^{w/4}$, we get a four-pass linear sort. ■

Exercises

10.39 Suppose that an input file consists of 1000 copies of each of the numbers 1 through 1000, each in a 32-bit word. Describe how you would take advantage of this knowledge to get a fast radix sort.

10.40 Suppose that an input file consists of 1000 copies of each of a thousand different 32-bit numbers. Describe how you would take advantage of this knowledge to get a fast radix sort.

10.41 What is the total number of bytes examined by three-way radix quicksort when sorting fixed-length bytestrings, in the worst case?

10.42 Empirically compare the number of bytes examined by three-way radix quicksort for long strings with $N = 10^3, 10^4, 10^5$, and 10^6 with the number of comparisons used by standard quicksort for the same files.

- **10.43** Give the number of bytes examined by MSD radix sort and three-way radix quicksort for a file of N keys A, AA, AAA, AAAA, AAAAA, AAAAAA, ...

10.7 Sublinear-Time Sorts

The primary conclusion that we can draw from the analytic results of Section 10.6 is that the running time of radix sorts can be sublinear in the total amount of information in the keys. In this section, we consider practical implications of this fact.

The LSD radix-sort implementation given in Section 10.5 makes `bytesword` passes through the file. By making R large, we get an efficient sorting method, as long as N is also large and we have space for R counters. As mentioned in the proof of Property 10.5, a reasonable choice is to make $\lg R$ (the number of bits per byte) about one-quarter of the word size, so that the radix sort is four key-indexed counting passes. Each byte of each key is examined, but there are only four digits per key. This example corresponds directly to the architectural organization of many computers: one typical organization has 32-bit words, each consisting of four 8-bit bytes. We extract bytes, rather than bits, from words, which approach is likely to be much more efficient on many computers. Now, each key-indexed-counting pass is linear, and, because there are only four of them, the entire sort is linear—certainly the best performance we could hope for in a sort.

In fact, it turns out that we can get by with only two key-indexed counting passes. We do so by taking advantage of the fact that the file will be *almost* sorted if only the leading $w/2$ bits of the w -bit keys are used. As we did with quicksort, we can complete the sort efficiently by using insertion sort on the whole file afterward. This method is a trivial modification to Program 10.4. To do a right-to-left sort using the leading one-half of the keys, we simply start the outer for loop at `bytesword/2-1`, rather than `bytesword-1`. Then, we use a conventional insertion sort on the nearly ordered file that results. Figures 10.3 and 10.18 provide convincing evidence that a file sorted on its leading bits is well ordered. Insertion sort would use only six exchanges to sort the file in the fourth column of Figure 10.3,

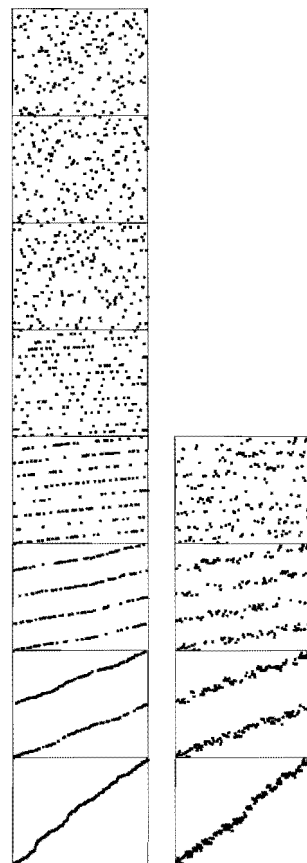


Figure 10.18
Dynamic characteristics of
LSD radix sort on MSD
bits

When keys are random bits, sorting the file on the leading bits of the keys brings it nearly into order. This diagram compares a six-pass LSD radix sort (left) on a file of random 6-bit keys with a three-pass LSD radix sort, which can be followed by an insertion-sort pass (right). The latter strategy is nearly twice as fast.

and Figure 10.18 shows that a larger file sorted on only the leading one-half of its bits also could be sorted efficiently by insertion sort.

For some file sizes, it might make sense to use the extra space that would otherwise be used for the auxiliary array to try to get by with just *one* key-indexed-counting pass, doing the rearrangement in place. For example, sorting 1 million random 32-bit keys could be done with one key-indexed-counting sort on the leading 20 bits, then an insertion sort. To do that, we need space just for the (1 million) counters—significantly less than would be needed for an auxiliary array. Using this method is equivalent to using standard MSD radix sort with $R = 2^{20}$, although it is essential that a small radix be used for small files for such a sort (see the discussion after Property 10.4).

The LSD approach to radix sorting is widely used, because it involves extremely simple control structures and its basic operations are suitable for machine-language implementation, which can directly adapt to special-purpose high-performance hardware. In such an environment, it might be fastest to run a full LSD radix sort. If we use pointers, then we need to have space for N links (and R counters) to use LSD radix sort, and this investment yields a method that can sort random files with only three or four passes.

In conventional programming environments, the inner loop of the key-indexed-counting program on which the radix sorts are based contains a substantially higher number of instructions than do the inner loops of quicksort or mergesort. This property of the implementations implies that the sublinear methods that we have been describing may not be as much faster than quicksort (say) as we might expect in many situations.

General-purpose algorithms such as quicksort are more widely used than radix sort, because they adapt to a broader variety of applications. The primary reason for this state of affairs is that the key abstraction on which radix sort is built is less general than the one that we used (with the compare function) in Chapters 6 through 9. For example, one typical way to arrange the interface for a sort utility is to have the client provide the comparison function. This is the interface used by the C library `qsort`. This arrangement not only handles situations where the client can use specialized knowledge about complex keys to implement a fast comparison, but also allows us to sort using

Table 10.1 Empirical study of radix sorts (integer keys)

These relative timings for radix sorts on random files of N 32-bit integers (all with a cutoff to insertion sort for N less than 16) indicate that radix sorts can be among the fastest sorts available, used with care. If we use a huge radix for tiny files, we ruin the performance of MSD radix sort, but adapting the radix to be less than the file size cures this problem. The fastest method for integer keys is LSD radix sort on the leading one-half of the bits, which we can speed up further by paying careful attention to the inner loop (see Exercise 10.45).

N	Q	4-bit bytes		8-bit bytes			16-bit bytes		
		M	L	M	L	L^*	M	L	M^*
12500	2	7	11	28	4	2	52	5	8
25000	5	14	21	29	8	4	54	8	15
50000	10	49	43	35	18	9	58	15	39
100000	21	77	92	47	39	18	67	30	77
200000	49	133	185	72	81	39	296	56	98
400000	102	278	377	581	169	88	119398	110	297
800000	223	919	732	6064	328	203	1532492	219	2309

Key:

- Q Quicksort, standard (Program 7.1)
- M MSD radix sort, standard (Program 10.2)
- L LSD radix sort (Program 10.4)
- M^* MSD radix sort, radix adapting to file size
- L^* LSD radix sort on MSD bits

an ordering relation that may not involve keys at all. We examine such an algorithm in Chapter 21.

When any of them could be used, the choice among quicksort and the various radix sort algorithms (and related versions of quicksort!) that we have considered in this chapter will depend not only on features of the application (such as key, record, and file size) but also on features of the programming and machine environment that relate to the efficiency of access and use of individual bits and bytes. Tables 10.1 and 10.2 give empirical results in support of the conclusion that the linear- and sublinear-time performance results that we have

Table 10.2 Empirical study of radix sorts (string keys)

These relative timings for various sorts on the first N words of *Moby Dick* (all, except heapsort, with a cutoff to insertion sort for N less than 16) indicate that the MSD-first approach is effective for string data. The cutoff for small subfiles is less effective for three-way radix quicksort than for the other methods, and is not effective at all unless we modify the insertion sort to avoid going through the leading parts of the keys (see Exercise 10.46).

N	Q	T	M	F	R	X	X*
12500	7	6	9	9	8	6	5
25000	14	12	18	19	15	11	10
50000	34	26	39	49	34	25	24
100000	83	61	87	114	71	57	54

Key:

- Q Quicksort, standard (Program 7.1)
- T Quicksort with three-way partitioning (Program 7.5)
- M Mergesort (Program 8.2)
- F Heapsort with Floyd's improvement (see Section 9.4)
- R MSD radix sort (Program 10.2)
- X Three-way radix quicksort (Program 10.3)
- X* Three-way radix quicksort (with cutoff)

discussed for various applications of radix sorts make these sorting methods an attractive choice for a variety of suitable applications.

Exercises

- ▷ 10.44 What is the major drawback of doing LSD radix sorting on the leading bits of the keys, then cleaning up with insertion sort afterward?
- 10.45 Develop an implementation of LSD radix sort for 32-bit keys with as few instructions as possible in the inner loop.
- 10.46 Implement three-way radix quicksort such that the insertion sort for small files does not use leading bytes that are known to be equal in comparisons.
- 10.47 Given 1 million random 32-bit keys, find the choice of byte size that minimizes the total running time when we use the method of using LSD radix sort on the first two bytes, then using insertion sort to clean up.

10.48 Answer Exercise 10.47 for 1 billion 64-bit keys.

10.49 Answer Exercise 10.48 for *three-pass* LSD radix sort.

Special-Purpose Sorting Methods

SORTING METHODS ARE critical components of many applications systems, and it is not unusual for special measures to be taken to make a sort as fast as possible or capable of handling huge files. We might encounter high-performance enhancements to a computer system, or special-purpose hardware specifically designed for sorting, or simply a new computer system based on some new architectural design. In such cases, the implicit assumptions that we have been making about the relative costs of operations on the data to be sorted may not be valid. In this chapter, we examine examples of sorting methods that are designed to run efficiently on various different kinds of machines. We consider several different examples of the restrictions imposed by high-performance hardware, and several methods that are useful in practice for implementing high-performance sorts.

Any new computer architecture is eventually going to need to support an efficient sorting method. Indeed, sorting has historically served as one testbed for evaluating new architectures, because it is so important and so well understood. We want to learn not just which known algorithms run best on a new machine and why, but also whether specific characteristics of a new machine can be exploited in some new algorithm. To develop a new algorithm, we define an abstract machine that encapsulates the essential properties of the real machine; design and analyze algorithms for the abstract machine; then implement, test, and refine both the best algorithms and the model. We draw on our past experience, including the many methods for general-purpose machines that we have seen in Chapters 6 through 10, but the abstract machines impose limitations that help us to focus on the true

costs, and make it clear that different algorithms are appropriate for different machines.

At one end of the spectrum, we shall consider low-level models where the only allowed operation is the compare–exchange operation. At the other end of the spectrum, we shall consider high-level models where we read and write large blocks of data to a slow external medium or among independent parallel processors.

First, we examine a version of mergesort known as *Batcher’s odd–even mergesort*. It is based on a divide-and-conquer merge algorithm that uses only compare–exchange operations, with *perfect-shuffle* and *perfect-unshuffle* operations for data movement. These are of interest in their own right, and apply to many problems other than sorting. Next, we examine Batcher’s method as a sorting network. A *sorting network* is a simple abstraction for low-level sorting hardware. Networks consist of interconnected *comparators*, which are modules capable of performing compare–exchange operations.

Another important abstract sorting problem is the *external-sorting* problem, where the file to be sorted is far too large to fit in memory. The cost of accessing individual records can be prohibitive, so we shall use an abstract model, where records are transferred to and from external devices in large blocks. We consider two algorithms for external sorting, and use the model to compare them.

Finally, we consider *parallel sorting*, for the case when the file to be sorted is distributed among independent parallel processors. We define a simple parallel-machine model, then examine how Batcher’s method provides an effective solution. Our use of the same basic algorithm to solve a high-level problem and a low-level problem is a convincing example of the power of abstraction.

The different abstract machines in this chapter are simple, but are worthy of study because they encapsulate specific constraints that can be critical in particular sorting applications. Low-level sorting hardware has to consist of simple components; external sorts generally require access of huge data files in blocks, with sequential access more efficient than random access; and parallel sorting involves communications constraints among processors. On the one hand, we cannot do justice to detailed machine models that fully correspond to particular real machines; on the other hand, the abstractions that we do consider lead us not only to theoretical formulations that provide

information about essential limitations on performance, but also to interesting algorithms that are of direct practical utility.

11.1 Batcher's Odd-Even Mergesort

To begin, we shall consider a sorting method that is based on just two abstract operations, the *compare-exchange* operation and the *perfect shuffle* operation (along with its inverse, the *perfect unshuffle*). The algorithm, developed by Batcher in 1968, is known as *Batcher's odd-even mergesort*. It is a simple task to implement the algorithm using shuffles, compare-exchanges, and double recursion, but it is more challenging to understand why the algorithm works, and to untangle the shuffles and recursion to see how it operates at a low level.

We encountered the compare-exchange operation briefly in Chapter 6, where we noted that some of the elementary sort methods discussed there could be expressed more concisely in terms of this abstract operation. Now, we are interested in methods that examine the data *exclusively* with compare-exchange operations. Standard comparisons are ruled out: The compare-exchange operation does not return a result, so there is no way for a program to take action that depends on data values.

Definition 11.1 A *nonadaptive sorting algorithm* is one where the sequence of operations performed depends on only the number of the inputs, rather than on the values of the keys.

In this section, we do allow operations that unilaterally rearrange the data, such as exchanges and perfect shuffles, but they are not essential, as we shall see in Section 11.2. Nonadaptive methods are equivalent to *straight-line programs* for sorting: They can be expressed simply as a list of the compare-exchange operations to be performed. For example, the sequence

```
compexch(a[0], a[1])  
compexch(a[1], a[2])  
compexch(a[0], a[1])
```

is a straight-line program for sorting three elements. We use loops, shuffles, and other high-level operations for convenience and economy in expressing algorithms, but our goal in developing an algorithm is to define, for each N , a fixed sequence of compexch operations that

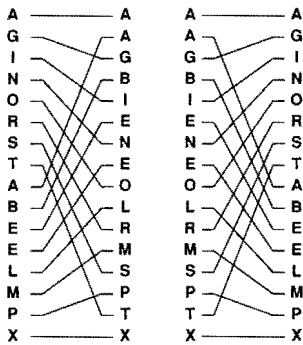


Figure 11.1
Perfect shuffle and perfect unshuffle

To perform a perfect shuffle (left), we take the first element in the file, then the first element in the second half, then the second element in the file, then the second element in the second half, and so forth. Consider the elements to be numbered starting at 0, top to bottom. Then, elements in the first half go to even-numbered positions, and elements in the second half go to odd-numbered positions. To perform a perfect unshuffle (right), we do the opposite: Elements in even-numbered positions go to the first half, and elements in odd-numbered positions go to the second half.

Program 11.1 Perfect shuffle and perfect unshuffle

The `shuffle` function rearranges a subarray `a[l], ..., a[r]` by splitting that subarray in half, then alternating elements from each half: Elements in the first half go in the even-numbered positions in the result, and elements in the second half go in the odd-numbered positions in the result.

The `unshuffle` function does the opposite: Elements in the even-numbered positions go in the first half of the result, and elements in the odd-numbered positions go in the second half of the result. We use these functions only for subarrays with an even number of elements.

```
shuffle(itemType a[], int l, int r)
{ int i, j, m = (l+r)/2;
  for (i = l, j = 0; i <= r; i+=2, j++)
    { aux[i] = a[l+j]; aux[i+1] = a[m+1+j]; }
  for (i = l; i <= r; i++) a[i] = aux[i];
}

unshuffle(itemType a[], int l, int r)
{ int i, j, m = (l+r)/2;
  for (i = l, j = 0; i <= r; i+=2, j++)
    { aux[l+j] = a[i]; aux[m+1+j] = a[i+1]; }
  for (i = l; i <= r; i++) a[i] = aux[i];
}
```

can sort any set of N keys. We can assume without loss of generality that the key values are the integers 1 through N (see Exercise 11.4); to know that a straight-line program is correct, we have to prove that it sorts each possible permutation of these values (see, for example, Exercise 11.5).

Few of the sorting algorithms that we considered in Chapters 6 through 10 are nonadaptive—they all use less or examine the keys in other ways, then take differing actions depending on key values. One exception is bubble sort (see Section 6.4), which uses only compare-exchanges. Pratt's version of shellsort (see Section 6.6) is another nonadaptive method.

Program 11.1 gives an implementation of the other abstract operations that we shall be using—the perfect shuffle and the perfect unshuffle—and Figure 11.1 gives an example of each. The perfect shuffle rearranges an array in a manner corresponding to the way that

Program 11.2 Batcher's odd-even merge (recursive version)

This recursive program implements an abstract inplace merge, using the `shuffle` and `unshuffle` operations from Program 11.1, although they are not essential—Program 11.3 is a bottom-up nonrecursive version of this program with shuffling removed. Our primary interest here is that this implementation provides a compact description of Batcher's algorithm, when the file size is a power of 2.

```
mergeTD(itemType a[], int l, int r)
{
    int i, m = (l+r)/2;
    if (r == l+1) compexch(a[l], a[r]);
    if (r < l+2) return;
    unshuffle(a, l, r);
    mergeTD(a, l, m);
    mergeTD(a, m+1, r);
    shuffle(a, l, r);
    for (i = l+1; i < r; i+=2)
        compexch(a[i], a[i+1]);
}
```

a deck of cards might be rearranged when shuffled by an expert: It is split precisely in half, then the cards are taken alternately from each half to make the shuffled deck. We always take the first card from the top half of the deck. If the number of cards is even, the two halves have the same number of cards; if the number of cards is odd, the extra card ends up in the top half. The perfect unshuffle does the opposite: We make the unshuffled deck by putting cards alternately in the top half and the bottom half.

Batcher's sort is exactly the top-down mergesort of Section 8.3; the difference is that instead of one of the adaptive merge implementations from Chapter 8, it uses Batcher's odd-even merge, a nonadaptive top-down recursive merge. Program 8.3 does not access the data at all, so our use of a nonadaptive merge implies that the whole sort is nonadaptive.

We shall implicitly assume in the text throughout this section and Section 11.2 that the number of items to be sorted is a power of 2. Then, we can always refer to " $N/2$ " without a caveat about N being odd, and so forth. This assumption is impractical, of course—our

```

A G I N O R S T A E E L M P X Y
A I O S A E M X G N R T E L P Y
A O A M I S E X
A A O M
A A
      M O
A M A O
A A M O
      I E S X
      E I
          S X
          E S I X
          E I S X
A E A I M S O X
A A E I M O S X
      G R E P N T L Y
      G E R P
      E G
          P R
      E P G R
      E G P R
          N L T Y
          L N
              T Y
              L T N Y
              L N T Y
          E L G N P T R Y
          E G L N P R T Y
A E A G E L I N M P O R S T X Y
A A E E G I L M N O P R S T X Y
```

Figure 11.2
Top-down Batcher's odd-even merge example

To merge `A G I N O R S T` with `A E E L M P X Y`, we begin with an unshuffle operation, which creates two independent merging problems of about one-half the size (shown in the second line): we have to merge `A I O S` with `A E M X` (in the first half of the array) and `G N R T` with `E L P Y` (in the second half of the array). After solving these subproblems recursively, we shuffle the solutions to these problems (shown in the next-to-last line) and complete the sort by compare-exchanging `E` with `A`, `G` with `E`, `L` with `I`, `N` with `M`, `P` with `O`, `R` with `S`, and `T` with `X`.

programs and examples involve other file sizes—but it simplifies the discussion considerably. We shall return to this issue at the end of Section 11.2.

Batcher's merge is itself a divide-and-conquer recursive method. To do a 1-by-1 merge, we use a single compare-exchange operation. Otherwise, to do an N -by- N merge, we unshuffle to get two $N/2$ -by- $N/2$ merging problems, and then solve them recursively to get two sorted files. Shuffling these files, we get a file that is nearly sorted—all that is needed is a single pass of $N/2 - 1$ independent compare-exchange operations: between elements $2i$ and $2i + 1$ for i from 1 to $N/2 - 1$. An example is depicted in Figure 11.2. From this description, the implementation in Program 11.2 is immediate.

Why does this method sort all possible input permutations? The answer to this question is not at all obvious—the classical proof is an indirect one that depends on a general characteristic of nonadaptive sorting programs.

Property 11.1 (0-1 principle) *If a nonadaptive program produces sorted output when the inputs are all either 0 or 1, then it does so when the inputs are arbitrary keys.*

See Exercise 11.7. ■

Property 11.2 *Batcher's odd-even merge (Program 11.2) is a valid merging method.*

Using the 0-1 principle, we check only that the method properly merges when the inputs are all either 0 or 1. Suppose that there are i 0s in the first subfile and j 0s in the second subfile. The proof of this property involves checking four cases, depending on whether i and j are odd or even. If they are both even, then the two merging subproblems each involve one file with $i/2$ 0s and one file with $j/2$ 0s, so both results have $(i + j)/2$ 0s. Shuffling, we get a sorted 0-1 file. The 0-1 file is also sorted after shuffling in the case that i is even and j is odd and the case that i is odd and j is even. But if both i and j are odd, then we end up shuffling a file with $(i + j)/2 + 1$ 0s with a file with $(i + j)/2 - 1$ 0s, so the 0-1 file after shuffling has $i + j - 1$ 0s, a 1, a 0, then $N - i - j - 1$ 1s (see Figure 11.3), and one of the comparators in the final stage completes the sort. ■

```

0 0 1 1 1 1 1 1 0 0 0 0 1 1 1 1
0 1 1 1 0 0 1 1 0 1 1 1 0 0 1 1
0 0 0 1 1 1 1 1 0 0 0 1 1 1 1 1
0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1
0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1

0 0 1 1 1 1 1 1 0 0 0 1 1 1 1 1
0 1 1 1 0 0 1 1 0 1 1 1 0 1 1 1
0 0 0 1 1 1 1 1 0 0 1 1 1 1 1 1
0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1
0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1

0 1 1 1 1 1 1 1 0 0 0 0 1 1 1 1
0 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1
0 0 0 1 1 1 1 1 0 0 1 1 1 1 1 1
0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1
0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1

0 0 0 1 1 1 1 1 0 0 0 1 1 1 1 1
0 0 1 1 0 0 1 1 0 1 1 1 0 1 1 1
0 0 0 0 1 1 1 1 0 0 1 1 1 1 1 1
0 0 0 0 0 1 0 1 1 1 1 1 1 1 1 1
0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1

```

Figure 11.3
Four cases for 0-1 merging

These four examples consist of five lines each: a 0-1 merging problem; the result of an unshuffle operation, which gives two merging problems; the result of recursively completing the merges; the result of a shuffle; and the result of the final odd-even compares. The last stage performs an exchange only when the number of 0s in both input files is odd.

We do not need actually to shuffle the data. Indeed, we can use Programs 11.2 and 8.3 to output a straight-line sorting program for any N , by changing the implementations of `compexch` and `shuffle` to maintain indices and to refer to the data indirectly (see Exercise 11.12). Or, we can have the program output the compare-exchange instructions to use on the original input (see Exercise 11.13). We could apply these techniques to any nonadaptive sorting method that rearranges the data with exchanges, shuffles, or similar operations. For Batchier's merge, the structure of the algorithm is so simple that we can develop a bottom-up implementation directly, as we shall see in Section 11.2.

Exercises

- ▷ 11.1 Give the result of shuffling and unshuffling the keys `EASYQUESTION`.
- 11.2 Generalize Program 11.1 to implement h -way shuffle and unshuffle. Defend your strategy for the case that the file size is not a multiple of h .
- 11.3 Implement the shuffle and unshuffle operations without using an auxiliary array.
- 11.4 Show that a straight-line program that sorts N distinct keys will sort N keys that are not necessarily distinct.
- ▷ 11.5 Show how the straight-line program given in the text sorts each of the six permutations of the integers 1, 2, and 3.
- 11.6 Give a straight-line program that sorts four elements.
- 11.7 Prove Property 11.1. *Hint:* Show that if the program does not sort some input array with arbitrary keys, then there is some 0–1 sequence that it does not sort.
- ▷ 11.8 Show how the keys `AEQSUYEINOST` are merged using Program 11.2, in the style of the example diagrammed in Figure 11.2.
- ▷ 11.9 Answer Exercise 11.8 for the keys `AESYEINOQSTU`.
- 11.10 Answer Exercise 11.8 for the keys `1001110000010100`.
- 11.11 Empirically compare the running time of Batchier's mergesort with that of standard top-down mergesort (Programs 8.3 and 8.2) for $N = 10^3$, 10^4 , 10^5 , and 10^6 .
- 11.12 Give implementations of `compexch`, `shuffle`, and `unshuffle` that cause Programs 11.2 and 8.3 to operate as an indirect sort (see Section 6.8).
- 11.13 Give implementations of `compexch`, `shuffle`, and `unshuffle` that cause Programs 11.2 and 8.3 to print out, given N , a straight-line program for sorting N elements. You may use an auxiliary global array to keep track of indices.

11.14 If we put the second file for the merge in reverse order, we have a *bitonic* sequence, as defined in Section 8.2. Changing the final loop in Program 11.2 to start at 1 instead of 1+1 turns the program into one that sorts bitonic sequences. Show how the keys A E S Q U Y T S O N I E are merged using this method, in the style of the example diagrammed in Figure 11.2.

- **11.15** Prove that the modified Program 11.2 described in Exercise 11.14 sorts any bitonic sequence.

11.2 Sorting Networks

The simplest model for studying nonadaptive sorting algorithms is an abstract machine that can access the data *only* through compare-exchange operations. Such a machine is called a *sorting network*. A sorting network comprises atomic *compare-exchange modules*, or *comparators*, which are wired together so as to implement the capability to perform fully general sorting.

Figure 11.4 shows a simple sorting network for four keys. Customarily, we draw a sorting network for N items as a sequence of N horizontal lines, with comparators connecting pairs of lines. We imagine that the keys to be sorted pass from right to left through the network, with a pair of numbers exchanged if necessary to put the smaller on top whenever a comparator is encountered.

Many details must be worked out before an actual sorting machine based on this scheme could be built. For example, the method of encoding the inputs is left unspecified. One approach would be to think of each wire in Figure 11.4 as a group of lines, each holding 1 bit of data, so that all the bits of a key flow through a line simultaneously. Another approach would be to have the comparators read their inputs 1 bit at a time along a single line (most significant bit first). Also left unspecified is the timing: mechanisms must be included to ensure that no comparator performs its operation before its input is ready. Sorting networks are a good abstraction because they allow us to separate such implementation considerations from higher-level design considerations, such as minimizing the number of comparators. Moreover, as we shall see in Section 11.5, the sort network abstraction is useful for applications other than direct circuit realizations.

Another important application of sorting networks is as a model for parallel computation. If two comparators do not use the same input lines, we assume that they can operate at the same time. For

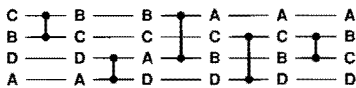


Figure 11.4
A sorting network

The keys move from left to right on the lines in the network. The comparators that they encounter exchange the keys if necessary to put the smaller one on the higher line. In this example, B and C are exchanged on the top two lines, then A and D are exchanged on the bottom two, then A and B, and so forth, leaving the keys in sorted order from top to bottom at the end. In this example, all the comparators do exchanges except the fourth one. This network sorts any permutation of four keys.

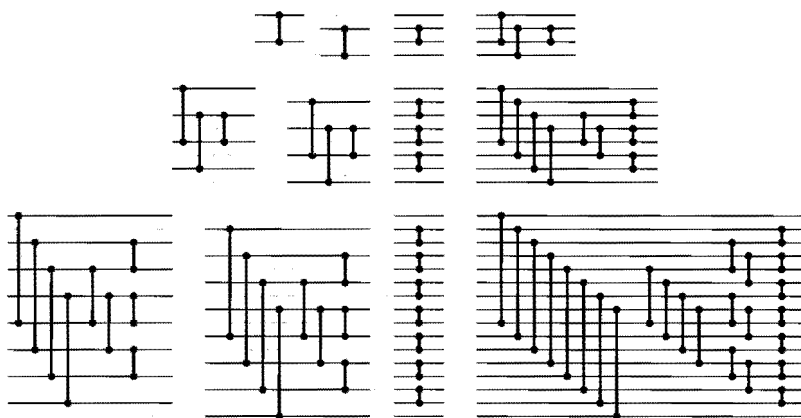


Figure 11.5
Batcher's odd-even merging networks

These different representations of the networks for four (top), eight (center), and 16 (bottom) lines expose the network's basic recursive structure. On the left are direct representations of the construction of the networks of size N with two copies of the networks of size $N/2$ (one for the even-numbered lines and one for the odd-numbered lines), plus a stage of comparators between lines 1 and 2, 3 and 4, 5 and 6, and so forth. On the right are simpler networks that we derive from those on the left by grouping comparators of the same length; grouping is possible because we can move comparators on odd lines past those on even lines without interference.

example, the network in Figure 11.4 shows that four elements can be sorted in three parallel steps. The 0–1 comparator and the 2–3 comparator can operate simultaneously in the first step, then the 0–2 comparator and the 1–3 comparator can operate simultaneously in the second step, and then the 2–3 comparator finishes the sort in the third step. Given any network, it is not difficult to classify the comparators into a sequence of *parallel stages* that consist of groups of comparators that can operate simultaneously (see Exercise 11.17). For efficient parallel computation, our challenge is to design networks with as few parallel stages as possible.

Program 11.2 corresponds directly to a merging network for each N , but it is also instructive for us to consider a direct bottom-up construction, which is illustrated in Figure 11.5. To construct a merging network of size N , we use two copies of the network of size $N/2$; one for the even-numbered lines and one for the odd-numbered lines. Because the two sets of comparators do not interfere, we can rearrange them to interleave the two networks. Then, at the end, we complete the network with comparators between lines 1 and 2, 3 and 4, and so forth. The odd-even interleaving replaces the perfect shuffle in Program 11.2. The proof that these networks merge properly is the same as that given for Properties 11.1 and 11.2, using the 0–1 principle. Figure 11.6 shows an example of the merge in operation.

Program 11.3 is a bottom-up implementation of Batchier's merge, with no shuffling, that corresponds to the networks in Figure 11.5. This program is a compact and elegant in-place merging function that

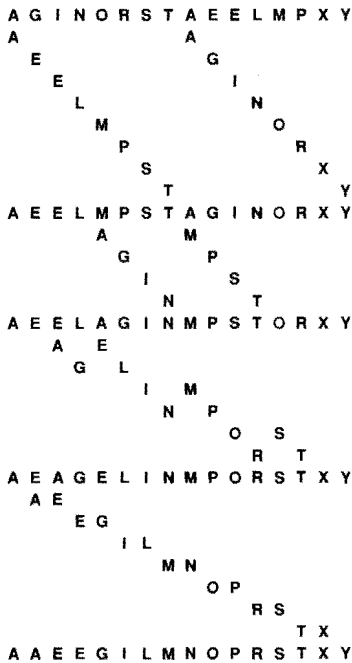


Figure 11.6
Bottom-up Batcher's merge
example

When all the shuffling is removed, Batcher's merge for our example amounts to the 25 compare-exchange operations depicted here. They divide into four phases of independent compare-exchange operations at a fixed offset for each phase.

Program 11.3 Batcher's odd-even merge (nonrecursive version)

This implementation of Batcher's odd-even merge (which assumes that the file size N is a power of 2) is compact but mysterious. We can understand how it accomplishes the merge by examining how it corresponds to the recursive version (see Program 11.2 and Figure 11.5). It accomplishes the merge in $\lg N$ passes consisting of uniform and independent compare-exchange instructions.

```
mergeBU(itemType a[], int l, int r)
{ int i, j, k, N = r-l+1;
  for (k = N/2; k > 0; k /= 2)
    for (j = k % (N/2); j+k < N; j += (k+k))
      for (i = 0; i < k; i++)
        compexch(a[l+j+i], a[l+j+i+k]);
}
```

is perhaps best understood as just an alternate representation of the networks, although direct proofs that it accomplishes the merging task correctly are also interesting to contemplate. We shall examine one such proof at the end of this section.

Figure 11.7 shows Batcher's odd-even sorting network, built from the merging networks in Figure 11.5 using the standard recursive mergesort construction. The construction is doubly recursive: once for the merging networks and once for the sorting networks. Although they are not optimal—we shall discuss optimal networks shortly—these networks are efficient.

Property 11.3 *Batcher's odd-even sorting networks have about $N(\lg N)^2/4$ comparators and can run in $(\lg N)^2/2$ parallel steps.*

The merging networks need about $\lg N$ parallel steps, and the sorting networks need $1 + 2 + \dots + \lg N$, or about $(\lg N)^2/2$ parallel steps. Comparator counting is left as an exercise (see Exercise 11.23). ■

Using the merge function in Program 11.3 within the standard recursive mergesort in Program 8.3 gives a compact in-place sorting method that is nonadaptive and uses $O(N(\lg N)^2)$ compare-exchange operations. Alternatively, we can remove the recursion from the mergesort and implement a bottom-up version of the whole sort directly, as shown in Program 11.4. As was Program 11.3, this program is per-

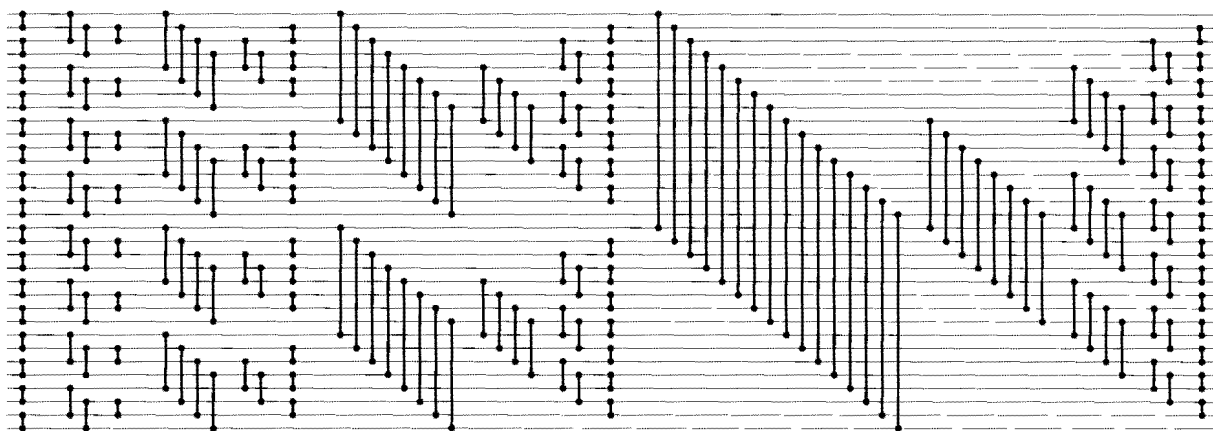


Figure 11.7
Batcher's odd-even sorting networks

This sorting network for 32 lines contains two copies of the network for 16 lines, four copies of the network for eight lines, and so forth. Reading from right to left, we see the structure in a top-down manner: A sorting network for 32 lines consists of a 16-by-16 merging network following two copies of the sorting network for 16 lines (one for the top half and one for the bottom half). Each network for 16 lines consists of an 8-by-8 merging network following two copies of the sorting network for 8 lines, and so forth. Reading from left to right, we see the structure in a bottom-up manner: The first column of comparators creates sorted subfiles of size 2; then, we have 2-by-2 merging networks that create sorted subfiles of size 4; then, 4-by-4 merging networks that create sorted subfiles of size 8, and so forth.

haps best understood as an alternate representation of the network in Figure 11.7. The implementation involves adding one loop and adding one test in Program 11.3, because the merge and the sort have similar recursive structure. To perform the bottom-up pass of merging a sequence of sorted files of length 2^k into a sequence of sorted files of length 2^{k+1} , we use the full merging network, but include only those comparators that fall completely within subfiles. This program perhaps wins the prize as the most compact nontrivial sort implementation that we have seen, and it is likely to be the method of choice when we want to take advantage of high-performance architectural features to develop a high-speed sort for small files (or to build a sorting network). Understanding how and why the program sorts would be a formidable task if we did not have the perspective of the recursive implementations and network constructions that we have been considering.

As usual with divide-and-conquer methods, we have two basic choices when N is not a power of 2 (see Exercises 11.24 and 11.21). We can divide in half (top-down) or divide at the largest power of 2 less than N (bottom-up). The latter is somewhat simpler for sorting networks, because it is equivalent to building a full network for the smallest power of 2 greater than or equal to N , then using only the first N lines and only comparators with both ends connected to those lines. The proof that this construction is valid is simple. Suppose that the lines that are not used have sentinel keys that are greater than any other keys on the network. Then, comparators on those lines never

Program 11.4 Batcher's odd-even sort (nonrecursive version)

This implementation of Batcher's odd-even sort corresponds directly to the network representation in Figure 11.7. It divides into phases, indexed by the variable p . The last phase, when p is N , is Batcher's odd-even merge. The next-to-last phase, when p is $N/2$, is the odd-even merge with the first stage and all comparators that cross $N/2$ eliminated; the third-to-last phase, when p is $N/4$, is the odd-even merge with the first two stages and all comparators that cross any multiple of $N/4$ eliminated, and so forth.

```
void batchersort(itemType a[], int l, int r)
{ int i, j, k, p, N = r-l+1;
  for (p = 1; p < N; p += p)
    for (k = p; k > 0; k /= 2)
      for (j = k%p; j+k < N; j += (k+k))
        for (i = 0; i < k; i++)
          if (j+i+k < N)
            if ((j+i)/(p+p) == (j+i+k)/(p+p))
              compexch(a[l+j+i], a[l+j+i+k]);
}
```

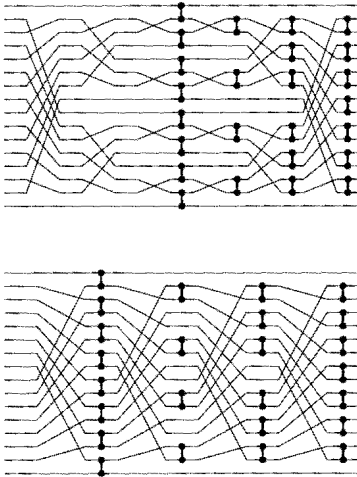


Figure 11.8
Shuffling in Batcher's odd-even merge

A direct implementation of Program 11.2 as a sorting network replete with recursive unshuffling and shuffling (top). An equivalent implementation (bottom) involves only full shuffles.

exchange, so removing them has no effect. Indeed, we could use *any* contiguous set of N lines from the larger network: Consider ignored lines at the top to have small sentinels and ignored lines at the bottom to have large sentinels. All these networks have about $N(\lg N)^2/4$ comparators.

The theory of sorting networks has an interesting history (*see reference section*). The problem of finding networks with as few comparators as possible was posed by Bose before 1960, and is called the *Bose-Nelson problem*. Batcher's networks were the first good solution to the problem, and for some time people conjectured that they were optimal. Batcher's *merging* networks *are* optimal, so any sorting network with substantially fewer comparators has to be constructed with an approach other than recursive mergesort. The problem of finding optimal sorting networks eluded researchers until, in 1983, Ajtai, Komlos, and Szemerédi proved the existence of networks with $O(N \log N)$ comparators. However, the AKS networks are a mathematical construction that is not at all practical, and Batcher's networks are still among the best available for practical use.

The connection between perfect shuffling and Batcher's networks makes it amusing to complete our study of sorting networks by considering yet another version of the algorithm. If we shuffle the lines in Batcher's odd-even merge, we get networks where all the comparators connect adjacent lines. Figure 11.8 illustrates a network that corresponds to the shuffling implementation corresponding to Program 11.2. This interconnection pattern is sometimes called a *butterfly network*. Also shown in the figure is another representation of the same straight-line program that provides an even more uniform pattern; it involves only full shuffles.

Figure 11.9 shows yet another interpretation of the method that illustrates the underlying structure. First, we write one file below the other; then, we compare those elements that are vertically adjacent and exchange them if necessary to put the larger one below the smaller one. Next, we split each row in half and interleave the halves, then perform the same compare-exchange operations on the numbers in the second and third lines. Comparisons involving other pairs of rows are not necessary because of the previous sorting. The split-interleave operation keeps both the rows and the columns of the table sorted. This property is preserved in general by the same operation: Each step doubles the number of rows, halves the number of columns, and still keeps the rows and the columns sorted; eventually we end up with 1 column of N rows, which is therefore completely sorted. The connection between the tableaux in Figure 11.9 and the network at the bottom in Figure 11.8 is that, when we write down the tables in column-major order (the elements in the first column followed by the elements in the second column, and so forth), we see that the permutation required to go from one step to the next is none other than the perfect shuffle.

Now, with an abstract parallel machine that has the perfect-shuffle interconnection built in, as shown in Figure 11.10, we would be able to implement directly networks like the one at the bottom of Figure 11.8. At each step, the machine does compare-exchange operations between some pairs of adjacent processors, as indicated by the algorithm, then performs a perfect shuffle of the data. Programming the machine amounts to specifying which pairs of processors should do compare-exchange operations at each cycle.

A E G G I M N R A B E E L M P X

A E G G I M N R A B E E I M N R
A B E E L M P X A E G G L M P X

A B E E A B E E
I M N R A E G G
A E G G I M N R
L M P X L M P X

A B A B
E E A E
A E E E
G G G G
I M I M
N R L M
L M N R
P X P X

A A
B A
A B
E E
E E
E E
G G
G G
I I
M L
L M
M M
N N
R P
P R
X X

Figure 11.9
Split-interleave merging

Starting with two sorted files in one row, we merge them by iterating the following operation: split each row in half and interleave the halves (left), and do compare-exchanges between items now vertically adjacent that came from different rows (right). At the beginning we have 16 columns and one row, then eight columns and two rows, then four columns and four rows, then two columns and eight rows, and finally 16 rows and one column, which is sorted.

Figure 11.10
A perfect shuffling machine

A machine with the interconnections drawn here could perform Batcher's algorithm (and many others) efficiently. Some parallel computers have connections like these.

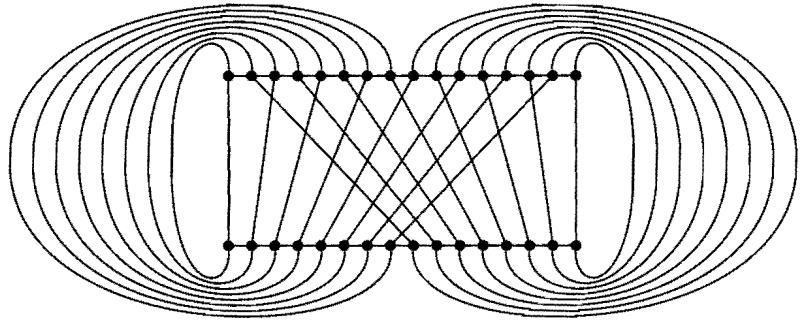


Figure 11.11 shows the dynamic characteristics of both the bottom-up method and this full-shuffling version of Batcher's odd-even merge.

Shuffling is an important abstraction for describing data movement in divide-and-conquer algorithms, and it arises in a variety of problems other than sorting. For example, if a 2^n -by- 2^n square matrix is kept in row-major order, then n perfect shuffles will transpose the matrix (convert the matrix to column-major order). More important examples include the fast Fourier transform and polynomial evaluation (see Part 8). We can solve each of these problems using a cycling perfect-shuffle machine like the one shown in Figure 11.10 but with more powerful processors. We might even contemplate having general-purpose processors that can shuffle and unshuffle (some real machines of this type have been built); we return to the discussion of such parallel machines in Section 11.5.

Exercises

- 11.16 Give sorting networks for four (see Exercise 11.6), five, and six elements. Use as few comparators as possible.
- 11.17 Write a program to compute the number of parallel steps required for any given straight-line program. *Hint:* Use the following labeling strategy. Label the input lines as belonging to stage 0, then do the following for each comparator: Label both output lines as inputs to stage $i + 1$ if the label on one of the input lines is i and the label on the other is not greater than i .
- 11.18 Compare the running time of Program 11.4 with that of Program 8.3, for randomly ordered keys with $N = 10^3, 10^4, 10^5$, and 10^6 .
- 11.19 Draw Batcher's network for doing a 10-by-11 merge.

- 11.20 Prove the relationship between recursive unshuffling and shuffling that is suggested by Figure 11.8.

- 11.21 From the argument in the text, there are 11 networks for sorting 21 elements hidden in Figure 11.7. Draw the one among these that has the fewest comparators.

11.22 Give the number of comparators in Batcher's odd-even sorting networks for $2 \leq N \leq 32$, where networks when N is not a power of 2 are derived from the first N lines of the network for the next largest power of 2.

- 11.23 For $N = 2^n$, derive an exact expression for the number of comparators used in Batcher's odd-even sorting networks. *Note:* Check your answer against Figure 11.7, which shows that the networks have 1, 3, 9, 25, and 65 comparators for N equal to 2, 4, 8, 16, and 32, respectively.

- 11.24 Construct a sorting network for sorting 21 elements using a top-down recursive style, where a network of size N is a composition of networks of sizes $\lfloor N/2 \rfloor$ and $\lceil N/2 \rceil$ followed by a merging network. (Use your answer from Exercise 11.19 as the final part of the network.)

11.25 Use recurrence relations to compute the number of comparators in sorting networks constructed as described in Exercise 11.24 for $2 \leq N \leq 32$. Compare your results with those that you obtained in Exercise 11.22.

- 11.26 Find a 16-line sorting network that uses fewer comparators than Batcher's network does.

11.27 Draw the merging networks corresponding to Figure 11.8 for bitonic sequences, using the scheme described in Exercise 11.14.

11.28 Draw the sorting network corresponding to shellsort with Pratt's increments (see Section 6.6), for $N = 32$.

11.29 Give a table containing the number of comparators in the networks described in Exercise 11.28 and the number of comparators in Batcher's networks, for $N = 16, 32, 64, 128$, and 256.

11.30 Design sorting networks that will sort files of N elements that are 3- and 4-sorted.

- 11.31 Use your networks from Exercise 11.30 to design a Pratt-like scheme based on multiples of 3 and 4. Draw your network for $N = 32$, and answer Exercise 11.29 for your networks.

- 11.32 Draw a version of Batcher's odd-even sorting network for $N = 16$ that has perfect shuffles between stages of independent comparators connecting adjacent lines. (The final four stages of the network should be those from the merging network at the bottom of Figure 11.8.)

- 11.33 Write a merging program for the machine in Figure 11.10, using the following conventions. An instruction is a sequence of 15 bits, where the i th bit, for $1 \leq i \leq 15$, indicates (if it is 1) that processor i and processor $i - 1$

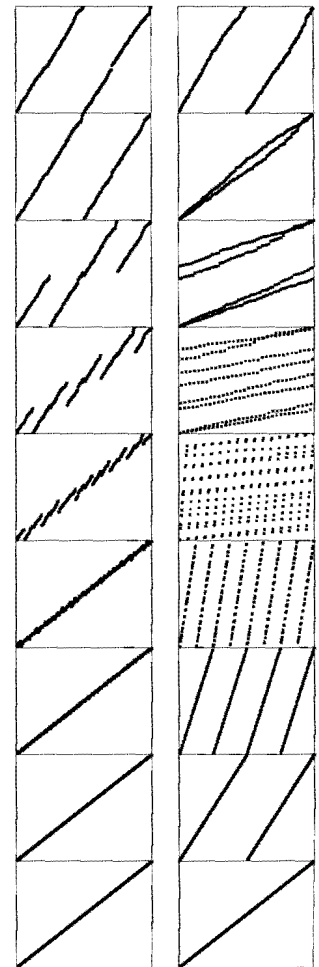


Figure 11.11
Dynamic characteristics of
odd-even merging

The bottom-up version of the odd-even merge (left) involves a sequence of stages where we compare-exchange the large half of one sorted subfile with the small half of the next. With full shuffling (right), the algorithm has an entirely different appearance.

should do a compare-exchange. A program is a sequence of instructions, and the machine executes a perfect shuffle between each instruction.

- 11.34 Write a sorting program for the machine in Figure 11.10, using the conventions described in Exercise 11.33.

11.3 External Sorting

We move next to another kind of abstract sorting problem, which applies when the file to be sorted is much too large to fit in the random-access memory of the computer. We use the term *external sorting* to describe this situation. There are many different types of external sorting devices, which can place a variety of different restrictions on the atomic operations used to implement the sort. Still, it is useful to consider sorting methods that use two basic primitive operations: *read* data from external storage into main memory, and *write* data from main memory onto external storage. We assume that the cost of these two operations is so much larger than the cost of primitive computational operations that we ignore the latter entirely. For example, in this abstract model, we ignore the cost of sorting the main memory! For huge memories or poor sorting methods, this assumption may not be justified; but it is generally possible to factor in an estimate of the true cost in practical situations if necessary.

The wide variety of types and costs of external storage devices makes the development of external sorting methods highly dependent on current technology. These methods can be complicated, and many parameters affect their performance; that a clever method might go unappreciated or unused because of a simple change in the technology is certainly a possibility in the study of external sorting. For this reason, we shall concentrate on reviewing general methods rather than on developing specific implementations in this section.

Over and above the high read-write cost for external devices, there are often severe restrictions on access, depending on the device. For example, for most types of devices, read and write operations between main memory and external storage are generally done most efficiently in large contiguous blocks of data. Also, external devices with huge capacities are often designed such that peak performance is achieved when we access the blocks in a *sequential* manner. For example, we cannot read items at the end of a magnetic tape without first

scanning through items at the beginning—for practical purposes, our access to items on the tape is restricted to those appearing somewhere close to the items most recently accessed. Several modern technologies have this same property. Accordingly, in this section, we concentrate on methods that read and write large blocks of data sequentially, making the implicit assumption that fast implementations of this type of data access can be achieved for the machines and devices that are of interest.

When we are in the process of reading or writing a number of different files, we assume that they are all on different external storage devices. On ancient machines, where files were stored on externally mounted magnetic tapes, this assumption was an absolute requirement. When working with disks, it is possible to implement the algorithms that we consider using only a single external device, but it generally will be much more efficient to use multiple devices.

A first step for someone planning to implement an efficient program to sort a huge file might be to implement an efficient program to make a copy of the file. A second step might be to implement a program to reverse the order of the file. Whatever difficulties arise in solving these tasks certainly need to be addressed in implementing an external sort. (The sort might have to do either one of them.) The purpose of using an abstract model is to allow us to separate such implementation issues from algorithm design issues.

The sorting algorithms that we examine are organized as a number of passes over all the data, and we usually measure the cost of an external sorting method by simply counting the number of such passes. Typically, we need relatively few passes—perhaps ten or fewer. This fact implies that eliminating even a single pass can significantly improve performance. Our basic assumption is that the running time of an external sorting method is dominated by input and output; thus, we can estimate the running time of an external sort by multiplying the number of passes it uses by the time required to read and write the whole file.

In summary, the abstract model that we shall use for external sorting involves a basic assumption that the file to be sorted is far too large to fit in main memory, and accounts for two other resources: running time (number of passes through the data) and the number of external devices available for use. We assume that we have

- N records to be sorted, on an external device
- space in the main memory to hold M records and
- $2P$ external devices for use during the sort.

We assign the the label 0 to the external device containing the input, and the labels 1, 2, $\dots$, $2P - 1$ to the others. The goal of the sort is to put the records back onto device 0, in sorted order. As we shall see, there is a tradeoff between P and the total running time—we are interested in quantifying that tradeoff so that we can compare competing strategies.

There are many reasons why this idealized model may not be realistic. Still, like any good abstract model, it does capture the essential aspects of the situation, and it does provide a precise framework within which we can explore algorithmic ideas, many of which are of direct utility in practical situations.

Most external sorting methods use the following general strategy. Make a first pass through the file to be sorted, breaking it up into blocks about the size of the internal memory, and *sort* these blocks. Then, *merge* the sorted blocks together, if necessary by making several passes through the file, creating successively larger sorted blocks until the whole file is sorted. This approach is called *sort-merge*, and it has been used effectively since computers first found widespread use in commercial applications in the 1950s.

The simplest sort-merge strategy, which is called *balanced multiway merging*, is illustrated in Figure 11.12. The method consists of an *initial distribution* pass, followed by several *multiway merging passes*.

In the initial distribution pass, we distribute the input among external devices $P, P + 1, \dots, 2P - 1$, in sorted blocks of M records each (except possibly the final block, which is smaller, if N is not a multiple of M). This distribution is easy to do—we read the first M records from the input, sort them, and write the sorted block onto device P ; then read the next M records from the input, sort them, and write the sorted block onto device $P + 1$; and so forth. If, after reaching device $2P - 1$ we still have more input (that is, if $N > PM$), we put a second sorted block on device P , then a second sorted block on device $P + 1$, and so forth. We continue in this way until the input is exhausted. After the distribution, the number of sorted blocks on each device is N/M rounded up or down to the next integer. If N is a multiple of M , then all the blocks are of size N/M (otherwise, all but

```

A S O R T I N G A N D M E R G I N G E X A M P L E W I T H F O R T Y F I V E R E C O R D S . $

A O S . D M N . A E X . F H T . E R V . $
I R T . E G R . L M P . O R T . C E O . $
A G N . G I N . E I W . F I Y . D R S . $

A A G I N O R S T . F F H I O R T T Y . $
D E G G I M N N R . C D E E O R R S V . $
A E E I L M P W X . $

A A A D E E E G G G I I I L M M N N N O P R R S T W X . $
C D E E F F H I O O R R R S T T V Y . $
. $

A A A C D D E E E E E F F G G G H I I I I L M M N N N O O O P R R R R R S S T T T V W X Y . $

```

the final one are of size N/M). For small N , there may be fewer than P blocks, and one or more of the devices may be empty.

In the first multiway merging pass, we regard devices P through $2P - 1$ as input devices, and devices 0 through $P - 1$ as output devices. We do P -way merging to merge the sorted blocks of size M on the input devices into sorted blocks of size PM , then distribute them onto the output devices in as balanced a manner as possible. First, we merge together the first block from each of the input devices and put the result onto device 0 ; then, we put the result of merging the second block on each input device onto device 1 ; and so forth. After reaching device $P - 1$, we put a second sorted block on device 0 , then a second sorted block on device 1 , and so forth. We continue in this way until the inputs are exhausted. After the distribution, the number of sorted blocks on each device is $N/(PM)$ rounded up or down to the next integer. If N is a multiple of PM , then all the blocks are of size PM (otherwise, the final block is smaller). If N is not larger than PM , there is just one sorted block left (on device 0), and we are finished.

Otherwise, we iterate the process and do a second multiway merging pass, regarding devices $0, 1, \dots, P - 1$ as the input devices, and devices $P, P + 1, \dots, 2P - 1$ as the output devices. We do P -way merging to make the sorted blocks of size PM on the input devices into sorted blocks of size P^2M , then distribute them back onto the output devices. We are finished after the second pass (with the result on device P) if N is not larger than P^2M .

Continuing in this way, back and forth between devices 0 through $P - 1$ and devices P through $2P - 1$, we increase the size of the blocks

Figure 11.12
Three-way balanced merge
example

In the initial distribution pass, we take the elements A S O from the input, sort them, and put the sorted run A O S on the first output device. Next, we take the elements R T I from the input, sort them, and put the sorted run I R T on the second output device. Continuing in this way, cycling through the output devices, we end with 15 runs: five on each output device. In the first merging phase, we merge A O S, I R T, and A G N to get A A G I O R S T, which we put on the first output device; then, we merge the second runs on the input devices to get D E G G I M N N R, which we put on the second output device; and so forth; again ending up with the data distributed in a balanced manner on three devices. We complete the sort with two additional merging passes.

0	1	2	3	4	5
15*1					
			5*1	5*1	5*1
2*3	2*3	1*3			
			1*9	1*6	
1*15					

Figure 11.13
Run distribution for balanced
3-way merge

In the initial distribution for a balanced three-way sort-merge of a file 15 times the size of the internal memory, we put five runs of relative size 1 on devices 3, 4, and 5, leaving devices 0, 1, and 2 empty. In the first merging phase, we put two runs of size 3 on devices 0 and 1, and one run of size 3 on device 2, leaving devices 3, 4, and 5 empty. Then, we merge the runs on devices 0, 1, and 2, and distribute them back to devices 3, 4, and 5, and so forth, continuing until only one run remains, on device 0. The total number of records processed is 60: four passes over all 15 records.

by a factor of P through P -way merges until we eventually have just one block, on device 0 or on device P . The final merge in each pass may not be a full P -way merge; otherwise the process is well balanced. Figure 11.13 depicts the process using only the numbers and relative sizes of the runs. We measure the cost of the merge by performing the indicated multiplications in this table, summing the results (not including the entry in the bottom row), and dividing by the initial number of runs. This calculation gives cost in terms of the number of *passes* over the data.

To implement P -way merging, we can use a priority queue of size P . We want to output repeatedly the smallest of the elements not yet output from each of the P sorted blocks to be merged, then to replace the element output with the next element from the block from which it came. To accomplish this action, we keep device indices in the priority queue, with a *less* function that reads the value of the key of the next record to be read from the indicated device (and provides a sentinel larger than all keys in records when the end of a block is reached). The merge is then a simple loop that reads the next record from the device having the smallest key and writes that record to the output, then replaces that record on the priority queue with the next record from the same device, continuing until a sentinel key is the smallest in the priority queue. We could use a heap implementation to make the time required for the priority queue proportional to $\log P$, but P is normally so small that this cost is dwarfed by the cost of writing to external storage. In our abstract model, we ignore priority-queue costs and assume that we have efficient sequential access to data on external devices, so that we can measure running time by counting the number of passes through the data. In practice, we might use an elementary priority-queue implementation and focus our programming on making sure that the external devices run at maximum efficiency.

Property 11.4 *With $2P$ external devices and internal memory sufficient to hold M records, a sort-merge that is based on a P -way balanced merge takes about $1 + \lceil \log_P(N/M) \rceil$ passes.*

One pass is required for distribution. If $N = MP^k$, the blocks are all of size MP after the first merge, MP^2 after the second, MP^3 after the third; and so forth. The sort is complete after $k = \log_P(N/M)$ passes. Otherwise, if $M^{P^{k-1}} < N < M^{P^k}$, the effect of incomplete and empty

blocks makes the blocks vary in size near the end of the process, but we are still finished after $k = \lceil \log_P(N/M) \rceil$ passes. ■

For example, if we want to sort 1 billion records using six devices and enough internal memory to hold 1 million records, we can do so with a three-way sort–merge with a total of eight passes through the data—one for distribution and $\lceil \log_3 1000 \rceil = 7$ merging passes. We will have sorted runs of 1 million records after the distribution pass, 3 million records after the first merge, 9 million records after the second merge, 27 million records after the third merge, and so forth. We can estimate that it should take about nine times as long to sort the file as it does to copy the file.

The most important decision to be made in a practical sort–merge is the choice of the value of P , the order of the merge. In our abstract model, we are restricted to sequential access, which implies that P has to be one-half the number of external devices available for use. This model is a realistic one for many external storage devices. For many other devices, however, nonsequential access is possible—it is just more expensive than sequential access. If only a few devices are available for the sort, nonsequential access might be unavoidable. In such cases, we can still use multiway merging, but we will have to take into account the basic tradeoff that increasing P will decrease the number of passes but increase the amount of (slow) nonsequential access.

Exercises

- ▷ 11.35 Show how the keys EASYQUESTIONWITHPLENTOFKEYS are sorted using 3-way balanced merging, in the style of the example diagrammed in Figure 11.12.
- ▷ 11.36 What would be the effect on the number of passes used in multiway merging if we were to double the number of external devices in use?
- ▷ 11.37 What would be the effect on the number of passes used in multiway merging if we were to increase by a factor of 10 the amount of internal memory available?
- 11.38 Develop an interface for external input and output that involves sequential transfer of blocks of data from external devices that operate asynchronously (or learn details about an existing one on your system). Use the interface to implement P -way merging, with P as large as you can make it while still arranging for the P input files and the input file to be on different output devices. Compare the running time of your program with the time required to copy the files to the output, one after another.

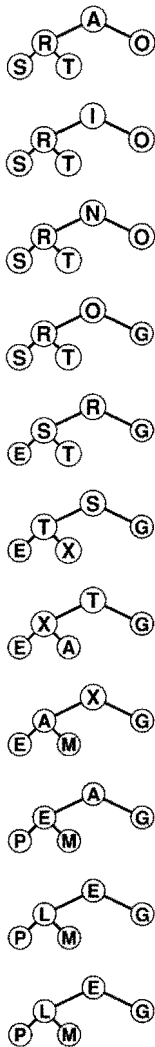


Figure 11.14
Replacement selection

This sequence shows how we can produce the two runs AINORSTX and AEEGLMP, which are of length 8 and 7, respectively, from the sequence ASORTINGEXAMPLE using a heap of size 5.

- 11.39 Use the interface from Exercise 11.38 to write a program to reverse the order of as large a file as is feasible on your system.
 - 11.40 How would you do a perfect shuffle of all the records on an external device?
 - 11.41 Develop a cost model for multiway merging that encompasses algorithms that can switch from one file to another on the same device, at a fixed cost that is much higher than the cost of a sequential read.
 - 11.42 Develop an external sorting approach that is based on partitioning a la quicksort or MSD radix sort, analyze it, and compare it with multiway merge. You may use a high level of abstraction, as we did in the description of sort-merge in this section, but you should strive to be able to predict the running time for a given number of devices and a given amount of internal memory.
- 11.43 How would you sort the contents of an external device if no other devices (except main memory) were available for use?
- 11.44 How would you sort the contents of an external device if only one extra device (and main memory) was available for use?

11.4 Sort-Merge Implementations

The general sort-merge strategy outlined in Section 11.3 is effective in practice. In this section, we consider two improvements that can lower the costs. The first technique, *replacement selection*, has the same effect on the running time as does increasing the amount of internal memory that we use; the second technique, *polyphase merging*, has the same effect as does increasing the number of devices that we use.

In Section 11.3, we discussed the use of priority queues for P -way merging, but noted that P is so small that fast algorithmic improvements are unimportant. During the initial distribution pass, however, we can make good use of fast priority queues to produce sorted runs that are longer than could fit in internal memory. The idea is to pass the (unordered) input through a large priority queue, always writing out the smallest element on the priority queue as before, and always replacing it with the next element from the input, with one additional proviso: If the new element is smaller than the one output most recently, then, because it could not possibly become part of the current sorted block, we mark it as a member of the next block and treat it as greater than all elements in the current block. When a marked element makes it to the top of the priority queue, we begin a new block. Figure 11.14 depicts the method in operation.

Property 11.5 *For random keys, the runs produced by replacement selection are about twice the size of the heap used.*

If we were to use heapsort to produce initial runs, we would fill the memory with records, then write them out one by one, continuing until the heap is empty. Then, we would fill the memory with another batch of records and repeat the process, again and again. On the average, the heap occupies only one-half the memory during this process. By contrast, replacement selection keeps the memory filled with the same data structure, so it is not surprising that it does twice as well. The full proof of this property requires a sophisticated analysis (*see reference section*), although the property is easy to verify experimentally (see Exercise 11.47). ■

For random files, the practical effect of replacement selection is to save perhaps one merging pass: Rather than starting with sorted runs about the size of the internal memory, then taking a merging pass to produce longer runs, we can start right off with runs about twice the size of the internal memory. For $P = 2$, this strategy would save precisely one merging pass; for larger P , the effect is less important. However, we know that practical sorts rarely deal with random files, and, if there is some order in the keys, then using replacement selection could result in huge runs. For example, if no key has more than M larger keys before it in the file, the file will be completely sorted by the replacement-selection pass, and no merging will be necessary! This possibility is the most important practical reason to use replacement selection.

The major weakness of balanced multiway merging is that only about one-half the devices are actively in use during the merges: the P input devices and whichever device is collecting the output. An alternative is always to do $(2P - 1)$ -way merges with all output onto device 0, then distribute the data back to the other tapes at the end of each merging pass. But this approach is not more efficient, because it effectively doubles the number of passes, for the distribution. Balanced multiway merging seems to require either an excessive number of tape units or excessive copying. Several clever algorithms have been invented that keep all the external devices busy by changing the way in which the small sorted blocks are merged together. The simplest of these methods is called *polyphase merging*.

```

A S O R T I N G A N D M E R G I N G E X A M P L E W I T H F O R T Y F I V E R E C O R D S . $
A O S . D M N . A E X . F H T . $
I R T . E G R . L M P . O R T . E R V . D R S . $
A G N . G I N . E I W . F I Y . C E O . . . $

A A G I N O R S T . D E G G I M N N R . A E E I L M P W X . F F H I O R T T Y . $
E R V . D R S . $
C E O . . . $

A A C E E G I N O O R R S T V . D D E G G I M N N R R S . $
A E E I L M P W X . F F H I O R T T Y . $
. $

D D E G G I M N N R R S . $
F F H I O R T T Y . $
A A A C E E E E G I I L M N O O P R R R S T V W X . $

A A A C D D E E E E E F F G G G H I I I I L M M N N N O O O P R R R R R S S T T T V W X Y . $

```

Figure 11.15
Polyphase merge example

In the initial distribution phase, we put the different numbers of runs on the tapes according to a prearranged scheme, rather than keeping the numbers of runs balanced, as we did in Figure 11.12. Then, we do three-way merges at every phase until the sort is complete. There are more phases than for the balanced merge, but the phases do not involve all the data.

The basic idea behind polyphase merging is to distribute the sorted blocks produced by replacement selection somewhat unevenly among the available tape units (leaving one empty) and then to apply a *merge-until-empty* strategy: Since the tapes being merged are of unequal length, one will run out sooner than the rest, and it then can be used as output. That is, we switch the roles of the output tape (which now has some sorted blocks on it) and the now-empty input tape, continuing the process until only one block remains. Figure 11.15 depicts an example.

The merge-until-empty strategy works for an arbitrary number of tapes, as shown in Figure 11.16. The merge is broken up into many *phases*, not all of which involve all of the data, and which involve no extra copying. Figure 11.16 shows how to compute the initial run distribution. We compute the *number* of runs on each device by working backward.

For the example depicted in Figure 11.16, we reason as follows: We want to finish the merge with 1 run, on device 0. Therefore, just before the last merge, we want device 0 to be empty, and we want to have 1 run on each of devices 1, 2, and 3. Next, we deduce the run distribution that we would need just before the next-to-last merge for that merge to produce this distribution. One of devices 1, 2, or 3 has to be empty (so that it can be the output device for the next-to-last merge)—we pick 3 arbitrarily. That is, the next-to-last merge merges